



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

INFORMÁCIÓS RENDSZEREK TANSZÉK

Fotográfiai témafelismerés neurális hálózatokkal nagy adathalmazon.

Témavezető:

Dr. Kiss Attila Elemér

Habilitált docens, tanszékvezető

Szerző:

Sándor Balázs

Programtervező informatikus MSc

Budapest, 2025

DIPLOMAMUNKA TÉMABEJELENTŐ

Hallgató adatai:

Név: Sándor Balázs

Neptun kód: AZA6NL

Képzési adatok:

Szak: programtervező informatikus, mesterképzés (MA/MSc)

Tagozat : Nappali

Belső témavezetővel rendelkezem

Témavezető neve: Dr. Kiss Attila Elemér

munkahelyének neve, tanszéke: ELTE IK, Információs rendszerek Tanszék

munkahelyének címe: 1117, Budapest, Pázmány Péter sétány 1/C.

beosztás és iskolai végzettsége: beosztás: habilitált docens tanszékvezető, végzettség: matematikus

A diplomamunka címe: Fotográfiai témafelismerés neurális hálózatokkal nagy adathalmazon.

A diplomamunka témája:

(A témavezetővel konzultálva adja meg 1/2 - 1 oldal terjedelemben diplomamunka témájának leírását)

Fotográfiai témafelismerés neurális hálózatokkal nagy adathalmazon.

A dolgozat témája, a fotográfiai témák neurális hálózatokkal történő felismerése.

Saját nagyméretű fotóadatbázisomból a neurális hálózatok tanításához szükséges címkézett, konvertált, átméretezett fotóadatbázist készítek és elkészítem ennek leíró statisztikáit, hogy összehasonlítható legyen más tanító adathalmazokkal.

Összegyűjtöm a témában született korábbi írásokban alkalmazott mérési módszereket és metrikákat melyek az összehasonlítás alapját képezik majd.

Osztályozási feladatként, szükséges a célosztályok meghatározása, melyet a korábbi írásokban alkalmazott osztályok és a lokális adathalmaz sajátosságainak összefűlésével készítek el.

A korábbi írások alapján összegyűjtöm a szabadon elérhető modelleket és témafelismerési eljárásokat majd ezeket alkalmazom a saját nagyméretű adatbázisomra és az említett metrikák alapján összehasonlítom őket, majd saját megoldást készítek a korábbiak felhasználásával és ezt a meghatározott mérőszámok szerint összevetem a meglévőkkel.

Az írás célja a megismert módszerek közül a meghatározott metrikák szerinti legjobb megoldás meghatározása, ezek kombinálása, fejlesztése és az új megoldás eredményeinek összevetése a korábbiakkal. Vizsgálat tárgya még a nagy adathalmaz hatásainak vizsgálata a meglévő modellek minőségére és performanciájára.

Budapest, 2024. 05. 22.

Tartalomjegyzék

1. Absztrakt	6
2. Bevezető	7
2.1. Motiváció	8
2.2. Elérhetőség	9
2.2.1. Adatok elérhetősége	9
2.2.2. Finomhangolt modellek elérhetősége	10
2.2.3. Programkódok elérhetősége	10
2.3. Mellékletek	11
2.3.1. Melléklet A: Mérési napló	12
2.3.2. Melléklet B: Az elkészített szoftverek	12
2.4. A feladat	14
2.4.1. Képosztályozás mint feladat	14
2.4.2. A konvolúciós neurális hálózatok mint megoldás	15
2.4.3. Képadatbázisok mint tanító eszközök	16
2.5. Irodalomkutatás	17
2.5.1. A metrikák	18
2.5.2. Áttekintő írások	18
2.5.3. A modellek és adathalmazok cikkei	18
2.5.4. Optimalizációs módszerek	19
2.5.5. Egyéb irodalom	19
3. Saját képadatbázis építése	20
3.1. A forrásképek	21
3.1.1. Motiváció	22
3.1.2. Források	23
3.2. A forrásadatbázis elemzése	24
3.2.1. Darabszámok és méretek	24

3.2.2.	Tematikus kategóriák	24
3.3.	Adatbázis tervezett szerkezete	25
3.3.1.	Képméret és formátum kiválasztása	26
3.3.2.	Osztályozási stratégia és kategóriarendszer kialakítása	27
3.3.3.	Mappaszerkezet kialakítása	28
3.4.	Előfeldolgozás	30
3.4.1.	Kézi előválogatás és kötegetlt adatgyűjtés	30
3.4.2.	Méret- és teljesítménykorlátok	30
3.5.	Saját feldolgozó futószalag	30
3.5.1.	Főbb jellemzők és technikai háttér	31
3.5.2.	A futószalag felépítése és működése	31
3.5.3.	A képfeldolgozó futószalag	32
3.5.4.	Konvertálás, Méretezés, Vágás	32
3.5.5.	Átnevezés és ismétlődések szűrése	34
3.5.6.	Az elkészült képek összemácsolása	36
3.6.	Kategorizálás és utófeldolgozás	37
3.6.1.	Automatikus osztályozás	37
3.6.2.	Places365 alapú kategorizálás	38
3.6.3.	ImageNet1K alapú kategorizálás	40
3.6.4.	Adatátviteli nehézségek	44
3.6.5.	Üres kategóriák eltávolítása	46
3.6.6.	Alkategóriák megszüntetése	46
3.6.7.	300 képnél kisebb kategóriák eltávolítása	47
3.6.8.	Kézi adattisztítás, szűrés és törlés	48
3.6.9.	Arcfelismerés alkalmazása	48
3.6.10.	OpenCV arcfelismerő szűrő.	48
3.6.11.	Nagy elemszámú kategóriák szétoosztása	49
3.6.12.	Felosztás és véglegesítés	49
3.7.	Kész adatbázis elemzése	50
3.8.	Statisztikák	50
3.8.1.	Darabszámok, méretek	50
3.8.2.	Mappavizsgáló szkript	50
3.8.3.	Kategóriák, statisztikák, méretek	50
3.8.4.	EXIF Statisztika szkript	50

3.8.5. EXIF statisztika	50
3.9. Az elkészült adatbázis értékelése	50
4. Modellek és metrikák kiválasztása	51
4.1. Metodika	52
4.2. Kiválasztott modellek	52
4.2.1. ResNet50	52
4.2.2. DenseNet121	52
4.2.3. EfficientNet-B0	52
4.2.4. ConvNeXt-Tiny	52
4.2.5. ViT-B/16	52
4.2.6. Inception-v3	52
4.2.7. NASNet	52
4.3. A kiválasztott metrikák:	52
4.3.1. Top-1 pontosság	52
4.3.2. Top-5 pontosság	52
4.3.3. Precision	52
4.3.4. Recall	52
4.3.5. F1-score	52
4.3.6. Train és Val loss	52
4.3.7. Train és Val pontosság	52
4.4. Összegzés	52
5. Vizsgálati metodika	53
5.1. I. Fázis: Tanítás nélküli tesztelés	53
5.2. II. Fázis: Teljes tanítás	53
5.3. III. Fázis: Iteratív tanítás	53
5.4. IV. Fázis: Saját modell fejlesztése	53
6. Szoftveres környezet	54
6.1. Lokális vagy Online	54
6.2. Google Colab lehetőségek	54
6.3. Tárolási problémák	54
6.4. Tanító és tesztelő programok	54
6.4.1. Tesztelés	54

6.4.2.	Tanítás	54
6.4.3.	Mérés	54
6.4.4.	Problémák és megoldások	54
6.5.	Összefoglalás	54
7.	I. Fázis: Tanítás nélküli tesztelés	55
7.1.	ResNet50	55
7.2.	DenseNet121	55
7.3.	EfficientNet-B0	55
7.4.	ConvNeXt-Tiny	55
7.5.	ViT-B/16	55
7.6.	Inception-v3	55
7.7.	NASNet	55
7.8.	Összefoglaló	55
8.	II. Fázis: Teljes tanítás	56
8.1.	ResNet50	56
8.2.	DenseNet121	56
8.3.	EfficientNet-B0	56
8.4.	ConvNeXt-Tiny	56
8.5.	ViT-B/16	56
8.6.	Inception-v3	56
8.7.	NASNet	56
8.8.	Összefoglaló	56
9.	III. Fázis: Iteratív tanítás	57
9.1.	ResNet50	57
9.2.	DenseNet121	57
9.3.	EfficientNet-B0	57
9.4.	ConvNeXt-Tiny	57
9.5.	ViT-B/16	57
9.6.	Inception-v3	57
9.7.	NASNet	57
9.8.	Összefoglaló	57

10. IV. Fázis: Saját modell fejlesztése	58
10.1. A ResNet18	58
10.2. A Base fej	58
10.3. A Deep fej	58
10.4. A Hybrid-v1 fej	58
10.5. A Hybrid-v2 fej	58
10.6. Mérések	58
10.6.1. I. Fázis: Tanítás nélküli tesztelés	58
10.6.2. II. Fázis: Teljes tanítás	58
10.6.3. III. Fázis: Iteratív tanítás	58
10.7. Összefoglalás	58
11. Legfontosabb eredmények	59
12. Összefoglaló	60
13. További lehetőségek	62
14. Befejezés	63
Irodalomjegyzék	64
Ábrajegyzék	67
Táblázatjegyzék	68
Forráskódjegyzék	69

1. fejezet

Absztrakt

A dolgozat célja egy nagyméretű, saját készítésű képadatbázis létrehozása és ezen mély neurális hálózatok osztályozási teljesítményének vizsgálata volt. Az SBphotothemes180k nevű adatbázis 180 750 egyedi, címkézett fényképet tartalmaz, kétféle annotációval: az ImageNet1K-ből szűrt 114, valamint a Places365-ből származó 121 kategóriával. Az osztályeloszlás erősen torzított, természetes adatok esetén tipikus, Long-tail eloszlást mutatott (aszimmetria: 8,4; szórás: 3200), de a nagy adatmennyiség és a szortírozási metodika révén kellően nagyra adódott az átlagos kategóriaméret is (1500). Az adatokon háromfázisú tesztelést végeztem hét népszerű, de különböző felépítésű neurális hálózaton (pl. ResNet50, EfficientNet-B0, ViT-B/16), különböző tanítóhalmaz-méretekkel. Az első lépés a tanítás nélküli modellek tesztelése volt a beépített súlyokkal, ezt követte a teljes tanítás, majd végül az iteratíván növekvő tanítóhalmazokon történő tesztelés. A modellek közül összességében az EfficientNet-B0 és a ConvNeXt-Tiny emelkedtek ki. A teljes adathalmazon tanított modellek közül az EfficientNet-B0 és a NASNet mutatták a legjobb teljesítményt (Top-1 pontosság 76%, Top-5 94%). Az utolsó fázisban a fokozatosan növelt tanítóhalmazokon mért teljesítményt vizsgáltam, hogy látható legyen a tanítóhalmaz méretének hatása. Ezen túl saját, erőforrástakarékos osztályozó modellt is fejlesztettem ResNet18-alapon, különféle fejrészekkel. Az egyszerű egyrétegű modell hasonló pontosságot ért el, mint az új modern fejrész, de az új modell számítási igénye jóval kisebb és pontossága összemérhető a nagy modellekével, így mobil eszközökön való alkalmazásra is alkalmas lehet. A nagyméretű adathalmaz, a többféle osztályozás, a nagy mennyiségű mért adat és a saját modellek további vizsgálatok alapját képezhetik a jövőben.

2. fejezet

Bevezető

Az utóbbi évtizedben a mesterséges intelligencia, különösen a mélytanulás (deep learning) területén tapasztalt fejlődés alapvetően átalakította a képfeldolgozás és számítógépes látás alkalmazásait. Az olyan modellek, mint a konvolúciós neurális hálózatok (CNN-ek), lehetővé tették a képek automatikus felismerését, osztályozását és tartalom szerinti csoportosítását, gyakran emberi szintű pontossággal. Az ilyen rendszerek sikere azonban nagymértékben függ a rendelkezésre álló nagy mennyiségű, jól címkézett adathalmazoktól. A legtöbb kutatás és alkalmazás olyan nyilvános benchmark adatkészleteken történik, mint az ImageNet vagy a Places365, amelyek széles körűek, de nem feltétlenül tükrözik a valós életből származó, természetes adatgyűjtés során keletkezett képek sokféleségét és osztályeloszlását. Jelen dolgozat célja egy valós, nem mesterségesen összeállított fotóadatbázison végzett osztályozási vizsgálat bemutatása. A saját készítésű, 20 év alatt összegyűlt fotógyűjteményemből kiindulva egy nagyméretű (180 750 képből álló), kétféleképpen címkézett adatbázist hoztam létre, amelyben minden kép egyedi, saját készítésű valós fotó, így ideális a természetes körülmények vizsgálatára. Az így előállított SBphotothemes180k adatbázison különböző népszerű neurális hálózatok (pl. ResNet, EfficientNet, Vision Transformer) osztályozási teljesítményét vizsgáltam többféle tanítási scenárióban, több metrika mentén. A dolgozat célja nem csupán a modellek pontosságának összehasonlítása, hanem annak feltárása, hogy milyen mértékben képesek a különböző architektúrák megbirkózni a természetes körülmények között gyűjtött adatok sokféleségével és a különféle tanítóhalmaz méretekkel. Emellett saját erőforrás-takarékos modell fejlesztésével is kísérletet tettem olyan alkalmazások számára, ahol a számítási teljesítmény korlátozott - például mobil vagy beágyazott rendszerek esetében.

2.1. Motiváció

A fényképezés számomra több mint hobbi: általános iskolás korom óta szerves része az életemnek. A fénykép egyrészt az emlékek megbízható őrzője, másrészt az önkifejezés egyedi formája is. Úgy gondolom, hogy egy jól elkészített fotó gyakran többet árul el az alkotóról, mint magáról a témáról. 2008 óta intenzíven foglalkozom fotózással, majd 2010-ben a Kalocsai Szent István Gimnázium stúdiójában, - amit Lakatos Ervin festőművész vezetett - mélyültem el a fényképezés művészi és tudatos megközelítésében. Ekkor kezdtem el a fotózást nem pusztán dokumentációs eszközként, hanem alkotói folyamatként gyakorolni. Azóta is aktívan fotózom, gyűjtöm, rendszerezem és archiválom a képeimet. A fotóim gyakran szorosan összekapcsolódnak más hobbijaimmal is: természetjárás, repülés, zene, csillagászat, valamint az állat- és növényvilág iránti érdeklődésem mind nyomot hagynak a képeimen. A vizualitás mindig is meghatározó szerepet játszott a gondolkodásomban. Programtervező informatikus alapszakos tanulmányaim során például háromdimenziós szimulációval foglalkoztam a szakdolgozatomban. A mesterképzésen - az információs rendszerek szakirányon - mélyebb betekintést nyerhettem a mesterséges intelligencia, azon belül is a mély neurális hálózatok működésébe és alkalmazási lehetőségeibe. A képzés során hamar szembesültem azzal, hogy a gépi tanulási rendszerek fejlesztésének egyik legnagyobb akadálya a minőségi, kellően címkézett adathalmazok hiánya. Ekkor tudatosult bennem, hogy az elmúlt évtizedek során saját magam által készített és gondosan karbantartott fotótár egyedülálló lehetőséget kínál egy valós, természetes környezetből származó adathalmaz kialakítására. Ez a felismerés adta a jelen diplomamunka ötletét: saját fotógyűjteményemet felhasználva egy neurális hálózatokkal végzett képosztályozási vizsgálatot hajtottam végre, amely egyszerre kapcsolódik szakmai érdeklődésemhez és személyes szenvedélyemhez is.

2.2. Elérhetőség

A munka során kiemelt figyelmet fordítottam arra, hogy minden lépést gondosan dokumentáljak, és az összes létrejött adatot – beleértve a képadatbázisokat, a finomhangolt modelleket, valamint az adatfeldolgozáshoz, tanításhoz és értékeléshez készített szoftvereket – átlátható és hozzáférhető módon elérhetővé tegyem.

Tekintettel arra, hogy a projekt során keletkezett fájlok összmérete meghaladja a 48,5 GB-ot, ezek megosztása az egyetemi OneDrive tárhelyem segítségével történt.

A projekt OneDrive mappájának elérhetősége:

https://ikelte-my.sharepoint.com/:f:/g/personal/sabuaai_inf_elte_hu/ElwbJQikJ7VJqM1u5lmhhNMBq0_falCiRZTuLwoimmdrpA?e=hGFCxI

Az elkészült szoftverek – köztük a Jupyter notebook-ok, Python és Bash szkriptek – a projekt GitHub tárhelyén érhetők el. A repository kisebb méretű, jól struktúrált, az összes a projekt során készült részletes dokumentációval ellátott fájlt tartalmazza.

Megjegyzés. A Jupyter notebook-ok megtekinthetők közvetlenül a GitHub felületén, vagy a Visual Studio Code segítségével is.

A projekt GitHub repository-jának elérhetősége:

<https://github.com/SandorBalazsHU/elte-ik-msc-thesis>

2.2.1. Adatok elérhetősége

A diplomamunkához készített és használt képadatbázisok, valamint a tanításhoz előkészített fájlok elérhetők a projekt dedikált OneDrive-mappájában. A megosztott tárhely struktúrált felépítésének célja, hogy a különböző feldolgozottsági szintű adatkészletek könnyen hozzáférhetők és újrahasznosíthatók legyenek.

A OneDrive-könyvtár főbb állományai:

- **`fine_tuned_models.zip`** (1,14 GB) – A különböző hálózati architektúrákon végzett teljes finomhangolás eredményeként létrejött modellek tömörített formában.
- **`sbphotothemes180k.zip`** (9,48 GB) – Az eredeti, kendezés nélküli, de már feldolgozott egyedi képeket tartalmazó képhalmaz, amely az SBphotothemes180k névre hallgató adatbázis alapját képezi.

- **sbphotothemes180k_imagenet114.zip** (9,48 GB) – Az SBphotothemes180k adatbázis ImageNet1K címkерendszer alapján szortírozott változata, 114 kategóriába rendezve.
- **sbphotothemes180k_imagenet114_split_80_10_10.zip** (9,48 GB) – Az előző adatbázis 80–10–10 százalékos arányban szétválasztott tanító, validáló és tesztelő halmazokra bontva, közvetlen tanításra alkalmas formában.
- **sbphotothemes180k_places121.zip** (9,48 GB) – A képhalmaz Places365 címkерendszer szerint 121 kategóriába rendezve.
- **sbphotothemes180k_places121_split_80_10_10.zip** (9,48 GB) – Az előző adatbázis tanításra előkészített változata, szintén 80–10–10 arányú Train–Val–Test bontásban.

2.2.2. Finomhangolt modellek elérhetősége

A projekt során minden, a teljes tanítóhalmazon végzett finomhangolás eredménye mentésre került, lehetővé téve a későbbi vizsgálatokat, összehasonlításokat és újrafelhasználást. Ezek a modellek szintén a OneDrive tárhelyen érhetők el:

- **fine_tuned_models.zip** (1,14 GB) – A zip-állomány a projekt során finomhangolt összes modell bináris fájljait tartalmazza.

A fájlnevek egységes és beszédes elnevezési konvenciót követnek, így egyértelműen beazonosítható, hogy az adott állomány melyik neurális hálózati modell és címkézési rendszer alapján végzett tanítás eredményeként készült.

2.2.3. Programkódok elérhetősége

A diplomamunka elkészítése során számos Jupyter notebook, Python- és Bash-alapú szkript készült, amelyek a projekt során végzett adatfeldolgozási, tanítási és kiértékelési lépéseket valósítják meg. Ezek a kódok átlátható és rendszerezett formában érhetők el a projekt GitHub-tárhelyén. A könyvtárszerkezet kialakításánál elsődleges szempont volt az olvashatóság, az átláthatóság és a könnyű navigáció.

A GitHub-repozitórium fő mappái az alábbiak:

- **literature** – Az irodalomkutatás során összegyűjtött szakirodalmi anyagokat tartalmazza.

- **notebooks** – A Google Colab környezethez készült notebookok, amelyek részletesen dokumentálják az elvégzett lépéseket. A programkód mellett az eredmények és megjegyzések is jól strukturáltan szerepelnek. Minden vizsgálati fázishoz külön mappa tartozik, valamint egy `db` nevű mappa az adatbázis építésének és elemzésének lépéseit tartalmazza. Az eredmények összegzésére szolgáló notebookok szintén itt találhatók.
- **scripts** – Ebben a mappában található az elkészült segédprogramok. A `cnn` alkönyvtár a neurális hálózati modellekhez és vizsgálatokhoz kapcsolódó kódokat tartalmazza, míg a `db` mappa az adatbázis előállításához használt szkripteket foglalja magában.
- **thesis-latex** – A diplomamunka szövegének LaTeX-alapú forráskódja, valamint a beillesztett ábrák találhatók itt.

A notebookok közvetlenül megtekinthetők a GitHub felületén, illetve Visual Studio Code vagy JupyterLab segítségével teljes funkcionalitással is futtathatók és szerkeszthetők.

A projekt során készült szoftverek részletes leírása és technikai bemutatása a *Melléklet B: Szoftverek* részben található.

2.3. Mellékletek

A dolgozat elkészítése során minden lépésnél tudatosan olyan szoftveres eszközöket fejlesztettem, amelyek lehetővé tették a mérésekhez kapcsolódó metrikák részletes és pontos rögzítését, valamint azok szemléletes vizualizációját. A keletkezett adatmennyiség és ábrák száma azonban jelentősen meghaladja a diplomamunka terjedelmi korlátait, ezért az eredményeket külön mellékletekben helyeztem el.

Az egyik melléklet a mérések teljes dokumentációját, nyers adatait és azok grafikus megjelenítését tartalmazza rendezett és jól áttekinthető formában. Egy másik mellékletben az általam készített szoftvereket mutatom be részletesen, külön figyelmet fordítva a fontosabb implementációs megoldásokra és az érdekesebb technikai megvalósításokra.

Az elkészült mellékletek a következők:

2.3.1. Melléklet A: Mérési napló

A mérési napló a diplomamunka mellékleteként szolgál, és a munka során végzett valamennyi mérés részletes dokumentációját tartalmazza. Az adatbázis felépítésének és struktúrájának ismertetése, a mérések pontos menete, az alkalmazott értékelési metrikák és a vizsgált modellek jellemzése a dolgozat vonatkozó fejezeteiben olvasható.

Ebben a mellékletben a mérések eredményei nyers, de rendezett formában kerültek rögzítésre. A metrikák (Top-1, Top-5 pontosság, Precision, Recall, F1-score stb.) mellett szerepelnek a modellek teljesítményének vizuális megjelenítései is, grafikonok és táblázatok formájában. A jobb áttekinthetőség érdekében minden mérési sorozat után rövid, szöveges összefoglaló is készült, amely kiemeli a legfontosabb tapasztalatokat és tendenciákat.

A mérési napló célja a teljes átláthatóság és reprodukálhatóság biztosítása, valamint az, hogy a részletes adatokhoz való hozzáférés segítse az eredmények értelmezését és további elemzését. A mérési napló kattintható tartalomjegyzékkel rendelkezik megkönnyítendő a navigálást és a keresett adatok, grafikonok gyors elérését.

2.3.2. Melléklet B: Az elkészített szoftverek

A diplomamunka során elkészített szoftverek központi szerepet játszottak az adatfeldolgozásban, a tanítási és tesztelési folyamatok automatizálásában, valamint a mért adatok kiértékelésében. A teljes munkafolyamat Python nyelven készült, Jupyter notebookok, PyTorch keretrendszer, valamint a matplotlib, seaborn, pandas és numpy könyvtárak használatával. A modellek betanítása és kiértékelése saját fejlesztésű kódbázison történt, amely lehetővé tette az egyedi igények szerinti metrika- és grafikon-előállítását.

A fejlesztett szoftvermodulok a következő főbb komponensekre bonthatók:

Adat-előkészítő és konvertáló eszközök: Ezek a programrészek a nyers fotóállományból készítettek előfeldolgozott, egységesített és annotált képadatbázist. Ide tartozik az EXIF-adatok megőrzését biztosító átalakítás, az MD hash alapú elnevezés és ismétlődés szűrés, az automatikus kategorizálás, a megfelelő képméretre vágás, tömörítés, valamint a tanító/tesztelő/validáló halmazok létrehozása.

A modellek tanítását vezérlő szkriptek: A tanítórendszer lehetővé teszi különféle modellek (ResNet50, DenseNet121, EfficientNet-B0, ConvNeXt-Tiny, stb.) betöltését,

finomhangolását, valamint többféle tanítási fázis (pl. teljes tanítóhalmaz, iteratív bővítés) automatizálását. A szkriptek rögzítik az összes fontos tanítási metrikát (loss, accuracy, precision, recall, F1-score), epochonként és osztályszinten.

Tesztelési és kiértékelő modulok: Ezek a programok lehetővé teszik a betanított vagy gyári súlyokkal rendelkező modellek teljesítményének összehasonlítását különféle metrikák alapján, mind Top-1, Top-5, mind osztályszpecifikus pontosság szerint. A program automatikusan készít confusion matrixokat, osztályeloszlás-görbéket, metrika-idő vagy metrika-mintaszám függvényeket.

Vizualizációs alrendszer: A metrikák kiértékelését követően a rendszer képes automatikusan egységes formátumú és jól olvasható grafikonokat generálni. A matplotlib és seaborn könyvtárakkal készült diagramok a tanítási folyamat vizuális dokumentációját nyújtják, megkönnyítve az összehasonlítást.

Egyedi fejlesztésű kis modell és fejrész prototípusok: A szoftvercsomag része az erőforrás-kímélő, ResNet18-alapú lokális modell fejlesztését támogató kódbázis is, amely lehetővé teszi különféle hálózati „fejek” tesztelését és kiértékelését. Ezek külön paramétereizhetők és egységes módon mérhetők a többi modellhez hasonlóan. Minden programrész jól dokumentált, verziókövetett formában készült, a főbb beállítások és paraméterek külön konfigurációs blokkokban találhatóak a notebook-okban. A virtualizált környezet kialakításáért felelős telepítő blokkok is elhelyezésre kerültek a notebook-okba a könnyebb megismételhetőség érdekében. A melléklet tartalmazza a szoftverrendszer logikai felépítésének leírását, a legfontosabb fájlok magyarázatát, valamint a kód működésének szemléltetését példákkal.

2.4. A feladat

A dolgozat célja egy saját készítésű, nagyméretű, természetes környezetben gyűjtött fotókból álló képadatbázis létrehozása és ezen különféle mély neurális hálózatok képosztályozási teljesítményének vizsgálata. A vizsgálat középpontjában az áll, hogy a valós, long-tail osztályeloszlással rendelkező adathalmazok milyen kihívásokat jelentenek a korszerű képfeldolgozó modellek számára, valamint hogy ezek a modellek milyen mértékben képesek adaptálódni az adatok mennyiségének és kiegyensúlyozatlanságának változásaihoz.

A dolgozat során egy 180 750 képből álló, saját fotóimból összeállított és kétféle címkézési sémával (ImageNet1K és Places365) ellátott adatbázist készítettem. Ezt követően több ismert mélytanulási architektúra – többek között ResNet50, EfficientNet-B0, ViT-B/16 és ConvNeXt-Tiny – teljesítményét értékeltem három fázisban: előtanítás nélküli, teljes tanításon átesett, valamint növekvő méretű tanítóhalmazon történő tanítás után. Az eredményeket több metrika mentén, például Top-1 és Top-5 pontosság, Precision, Recall és F1-score segítségével elemeztem.

Emellett célom volt egy saját, erőforrástakarékos modell megalkotása is, amely kis méretű, lokális alkalmazásokban – például mobil eszközökön – hatékonyan használható, miközben teljesítménye megközelíti a nagyobb méretű, általános célú modellekét.

2.4.1. Képosztályozás mint feladat

A képosztályozás (image classification) a számítógépes látás és képfeldolgozás egyik alapvető problémája, amelynek célja, hogy egy bemeneti képről automatikusan eldöntsük, melyik előre definiált osztályba tartozik. Ez a feladat különösen fontos számos gyakorlati alkalmazásban, például arcfelismerés, orvosi képelemzés, önvezető járművek környezeti értelmezése vagy digitális fotóarchívumok rendezése esetén. A klasszikus képosztályozási megközelítések során jellemzően előre definiált jellemzők (például élek, színek, textúrák) segítségével történt az osztályba sorolás. Azonban a mélytanulás, különösen a konvolúciós neurális hálózatok (CNN-ek) megjelenése forradalmasította ezt a területet: ezek a modellek képesek automatikusan megtanulni a képek releváns elemeit, így jelentősen javítva a pontosságot.

A képosztályozás során a neurális hálózat bemenete egy fix méretű kép, a kimenete pedig egy valószínűségi eloszlás az osztályok között, amelyből kiválasztható a leg-

valószínűbb címke. A feladat kihívásai közé tartozik az osztályok közti hasonlóság, a képi tartalom sokfélesége, a világítás, szögtorzítás, háttér és egyéb zavaró tényezők, valamint az osztályeloszlások kiegyensúlyozatlansága, amely különösen a valós adathalmazok esetén gyakori.

A long-tail eloszlás külön figyelmet igényel a képosztályozásban, mivel az adatok túlnyomó része néhány gyakori kategóriára koncentrálódik, míg a legtöbb osztály csak kisebb számú mintával szerepel de ezek összessége mégis jelentős arányt képvisel az adathalmazban. Az ilyen disztribúciók kezelése különösen nehéz feladat a tanuló modellek számára, mivel a hagyományos algoritmusok hajlamosak alultanulni a ritkább osztályokat.

A képosztályozási feladat tehát nem csupán technikai kihívás, hanem kulcsfontosságú építőelem sok mesterséges intelligencia alapú rendszer számára. A jelen dolgozatban bemutatott vizsgálatok célja, hogy feltérképezzék, hogyan viselkednek a különféle modern neurális hálózati architektúrák, amikor valós, nem szintetikus kiegyensúlyozott adatokkal kell megbirkózniuk.

Források: [1] [2] [3]

2.4.2. A konvolúciós neurális hálózatok mint megoldás

A konvolúciós neurális hálózatok (CNN-ek, azaz Convolutional Neural Networks) az utóbbi évtizedben a gépi látás és képfeldolgozás egyik legmeghatározóbb eszközévé váltak. A hagyományos, teljesen összekötött (fully connected) neurális hálózatokkal szemben a CNN-ek képesek kihasználni a képek térbeli struktúráját, és ennek köszönhetően hatékonyabban tanulnak vizuális jellemzőket.

A CNN-ek alapötlete a lokális jellemzők felismerésén alapszik: a bemeneti képen végighaladó konvolúciós szűrők képesek detektálni alacsony szintű mintázatokat, például éleket, sarkokat vagy textúrákat. Az egymásra épülő rétegek révén a hálózat egyre összetettebb vizuális absztrakciókat tanul meg, amelyek végső soron lehetővé teszik az osztályok közötti különbségek felismerését.

A konvolúciós rétegeket általában nemlineáris aktivációs függvények (például ReLU [4]) követik, melyek nemlinearitást visznek a rendszerbe, azaz segítik, hogy a háló ne csak lineáris műveleteket tanuljon meg, majd pooling rétegek [5] [6], amelyek egyrészt csökkentik a reprezentáció térbeli méretét, másrészt hozzájárulnak a jellemzők eltolás-invarianciájához (pl. ha a tárgy kis mértékben más helyen van a képen, a

pooling elősegíti, hogy a jellemzők ennek ellenére hasonlóak maradjanak), ezáltal növelve a modell általánosítási képességét. A hálózat végén egy vagy több teljesen összekötött réteg található, amely a tanult jellemzők alapján dönt az osztályokhoz való hozzárendelésről. A tanítási folyamat a hibák visszaterjesztésén (backpropagation) és gradiensalapú optimalizáción alapszik.

A CNN-ek sikerének egyik kulcsa az, hogy nem igényelnek kézzel definiált jellemzőkészleteket - a hálózat maga tanulja meg az optimális reprezentációkat a bemeneti adatokból. Ez különösen előnyös olyan esetekben, ahol a képek nagy változatosságot mutatnak, vagy ahol nem áll rendelkezésre szakértői tudás a jellemzők kézi kiválasztásához.

Az elmúlt évek során számos jól ismert CNN-alapú architektúra született (például LeNet, AlexNet, VGG, ResNet, DenseNet, EfficientNet, stb.), amelyek mind különféle technikai újításokat vezettek be a hálózatok mélysége, számítási hatékonysága vagy tanulási képességei terén. A jelen dolgozat több ilyen, különféle generációba tartozó modellt is vizsgál, hogy összehasonlíthatóvá váljon teljesítményük egy valós, természetes képekből álló, kiegyensúlyozatlan eloszlású adathalmazon.

Források: [7] [3] [1]

2.4.3. Képadatbázisok mint tanító eszközök

A gépi tanulási rendszerek, különösen a mély neurális hálózatok sikerességének egyik kulcsa a megfelelő mennyiségű és minőségű tanítóadat megléte. A képosztályozási feladatok esetén ez különösen fontos, hiszen a neurális hálózatok képesek milliányi paramétert megtanulni, de ehhez nagyméretű, változatos és jól strukturált képadatbázisokra van szükség. Az ilyen adathalmazok nemcsak a modellek hatékony tanítását teszik lehetővé, hanem azok általánosító képességét is alapvetően meghatározzák. Számos nyílt képadatbázis - mint például az ImageNet [8] a CIFAR-10, a MNIST vagy a Places365 [9] - alapvető szerepet játszott a mesterséges intelligencia fejlődésében, mivel ezek standardizált környezetet biztosítottak a modellek tanításához és összehasonlításához. Azonban ezek az adathalmazok gyakran laboratóriumi környezetben előállított, válogatott és sokszor mesterségesen kiegyensúlyozott mintákból állnak, ami korlátozza az általános életszerű alkalmazásokra való felkészítést.

A valós világ képei gyakran heterogének, torzítottak, zajosak, és az osztályok eloszlása erősen aszimmetrikus. Az ilyen „long-tail” eloszlású adatbázisok, ahol sok

kategória kevés példánnyal szerepel, komoly kihívást jelentenek a tanuló algoritmusok számára. Az ezekkel való munka viszont közelebb hozza a neurális modellek viselkedését a valódi alkalmazási környezetekhez, ahol az adatok ritkán ideálisak.

A dolgozatban bemutatott egyedi képadatbázis - az SBphotothemes180k - egy különleges erőforrás, mivel saját fotógyűjteményből származik, természetes módon jött létre, és hosszú időszak alatt, különféle helyszíneken, változatos körülmények között készült képeket tartalmaz. Ez a típusú adathalmaz ideális arra, hogy feltérképezzük, hogyan teljesítenek a neurális hálózatok nem laboratóriumi környezetből származó, organikus képadatokon.

Egy jól megtervezett képadatbázis nemcsak a tanítás során nyújt alapot, hanem lehetőséget teremt arra is, hogy vizsgáljuk a modellek robusztusságát, általánosítási képességét, valamint azt, hogyan reagálnak az adatok mennyiségi vagy minőségi változásaira. Ezért a képadatbázis nem csupán háttéranyag, hanem aktív és kulcsfontosságú szereplője a neurális hálózatokkal végzett kutatómunkának.

Források: [3] [7] [10]

2.5. Irodalomkutatás

A diplomamunka elkészítése során elsősorban a képzés során elsajátított elméleti és gyakorlati ismeretekre támaszkodtam, különösen a *Bevezetés a gépi tanulásba*, a *Gépi tanulás alapjai Python-ban* és a *Deep Learning* kurzusokon tanultakra. Ezek az alapok biztosították a technológiai és módszertani keretet a munka megvalósításához.

A projekt előkészítési szakaszában emellett alapos irodalomkutatást végeztem: áttekintettem a felhasznált modelleket bemutató tudományos publikációkat, témába vágó összefoglaló (survey) tanulmányokat, valamint a képosztályozás és neurális hálózatok alkalmazási lehetőségeit vizsgáló szakirodalmat. E fejezet célja ezen források rövid ismertetése, valamint a kutatási háttér bemutatása.

A felhasznált irodalmak részletes jegyzéke a dolgozat végén, illetve a projekt GitHub könyvtárában is elérhető.

2.5.1. A metrikák

Az osztályozási eredmények értékelésére szolgáló metrikák kiválasztásában és pontos értelmezésében jelentős segítséget nyújtott az *"Image Classification with Convolutional Neural Networks"* [3] című tanulmány. A cikk világos áttekintést ad az olyan kulcsfontosságú mutatókról, mint az *accuracy*, *precision*, *recall*, *F1-score*, illetve a *confusion matrix* szerepe a modellek teljesítményének objektív értékelésében.

2.5.2. Áttekintő írások

Az irodalomkutatást áttekintő művek keresésével és értelmezésével kezdtem.

- "A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects"[7]
- "Review of deep learning: concepts, CNN architectures, challenges, applications, future directions"[10]
- "Understanding Deep Convolutional Networks"[11]
- "Benchmarking Neural Network Training Algorithms"[12]

2.5.3. A modellek és adathalmazok cikkei

Természetesen nagyon nagy segítséget jelentettek a felhasznált modelleket és kategorizálásokat ismertető írások. Ezek az írások a következők:

- A ResNet50 modell ismertetése.[13]
- DenseNet121 modell ismertetése.[14]
- EfficientNet-B0 modell ismertetése.[15]
- ConvNeXt-Tiny modell ismertetése.[16]
- ViT-B/16 modell ismertetése.[17]
- Inception-v3 modell ismertetése.[18]
- NASNet modell ismertetése.[19]

- ImageNet1K osztályozás ismertetése.[8]
- Places365 osztályozás ismertetése.[9]

2.5.4. Optimalizációs módszerek

Amikor az új modellt terveztem több optimalizációs megoldásnak is utánanéztem. Az alábbiakban az ezekhez tartozó irodalmakat sorolom fel.

- GELU (Gaussian Error Linear Unit) aktivációs függvény[4]
- Layer Normalization[5]
- Dropout regulárizáció[6]
- Batch Normalization (BatchNorm)[20]
- Adaptive Average Pooling (AdaptiveAvgPool2d)[21]

2.5.5. Egyéb irodalom

A további cikkek és írások amiket áttekintettem a munka megkezdése előtt az alábbiak:

- "A Survey on Deep Transfer Learning and Beyond"[22]
- "Landscape photo classification mechanism for context-aware photography support system" [23]
- "Photo Content Classification Using Convolutional Neural Network" [2]
- "An Analysis Of Convolutional Neural Networks For Image Classification"[1]

3. fejezet

Saját képadatbázis építése

A mesterséges intelligencia alapú képosztályozás hatékony működésének alapfeltétele a megfelelő minőségű és kellően reprezentatív tanítóadatbázis. Jelen dolgozat egyik legfontosabb és legidőigényesebb eleme volt egy ilyen képadatbázis összeállítása, amely nemcsak mennyiségében, hanem tartalmában és szerkezetében is jól illeszkedik a konvolúciós neurális hálózatok igényeihez. Mivel rendelkezésemre állt egy saját fotóarchívum, amely két évtizednyi, gondosan rendszerezett és archivált felvételt tartalmaz, adódott a lehetőség, hogy egyéni képtáramból építsek fel egy teljes értékű osztályozási adatbázist.

A kiindulási anyag döntő többsége NEF (Nikon Electronic Format) típusú RAW formátumban volt elérhető, amely minden színcsatorna és világossági komponens részletes információit tartalmazza. Ez a formátum ideális alapot nyújt a professzionális feldolgozáshoz, ugyanakkor szükségessé tette egy egyedi konverziós pipeline megtervezését és megvalósítását. A cél az volt, hogy a képek átalakítása során a lehető legkevesebb információ vesszen el, ugyanakkor teljesüljön minden olyan követelmény, amely a neurális hálózatok betanításához szükséges: fix képméret, egységes képarány, szabványos formátum, egyértelmű osztályozás, ismétlődésmentes elrendezés és metaadatok megőrzése.

A teljes fotóarchívum mérete meghaladta a 2 terabájtot, így a képek feldolgozása szakaszosan, 100 GB-os csomagokban történt, a saját gépem SSD tárhelyének igénybevételével. Ez nemcsak a feldolgozási sebességet növelte, hanem csökkentette az archiváló háttértárolók elhasználódásának kockázatát is. A képfeldolgozó pipeline automatikusan hajtott végre a következő lépéseket: a NEF fájlok EXIF metaadatait megtartó konverzióját JPEG formátumba (80-as tömörítéssel), méretezést 224 kép-

pont szélességre, középre vágást 1:1 képarány eléréséhez, MD5-alapú fájlnev-képzést az ismétlődések kiszűrésére, valamint osztályozási és szűrési lépéseket. A kezdeti automatikus osztályozást többszörös kézi ellenőrzés és újraosztályozás követte, így fokozatosan alakult ki az adatbázis végleges struktúrája.

A kategóriák számát és tartalmát az osztályok kiegyensúlyozottságának elve, valamint a képanyag tartalmi homogenitása alapján határoztam meg. Az alacsony elemszámú osztályokat átcsoportosítottam, a túl nagy kategóriákat finomabb alkategóriákra bontottam, az irreleváns vagy túl heterogén osztályokat pedig elhagytam. A végső adatbázis több mint 180 000 képet tartalmaz, gondosan strukturált mappaszerkezetben, amely a tanító, validációs és tesztadatok 80-10-10 százalékos megosztását is tartalmazza.

A képadatbázis építése közben számos saját fejlesztésű segédprogram készült, amelyek automatizálták és nyomon követhetővé tették az egyes lépéseket. Ezek a programok nemcsak a képfeldolgozásért, hanem a statisztikai elemzésekért, vizualizációkért és a kategóriaeloszlás értékeléséért is felelősek. A fejezet hátralévő részében részletesen bemutatom az egyes lépéseket, a megvalósított pipeline-t, a feldolgozási logikát, valamint az e szakaszban keletkezett eredményeket és tapasztalatokat.

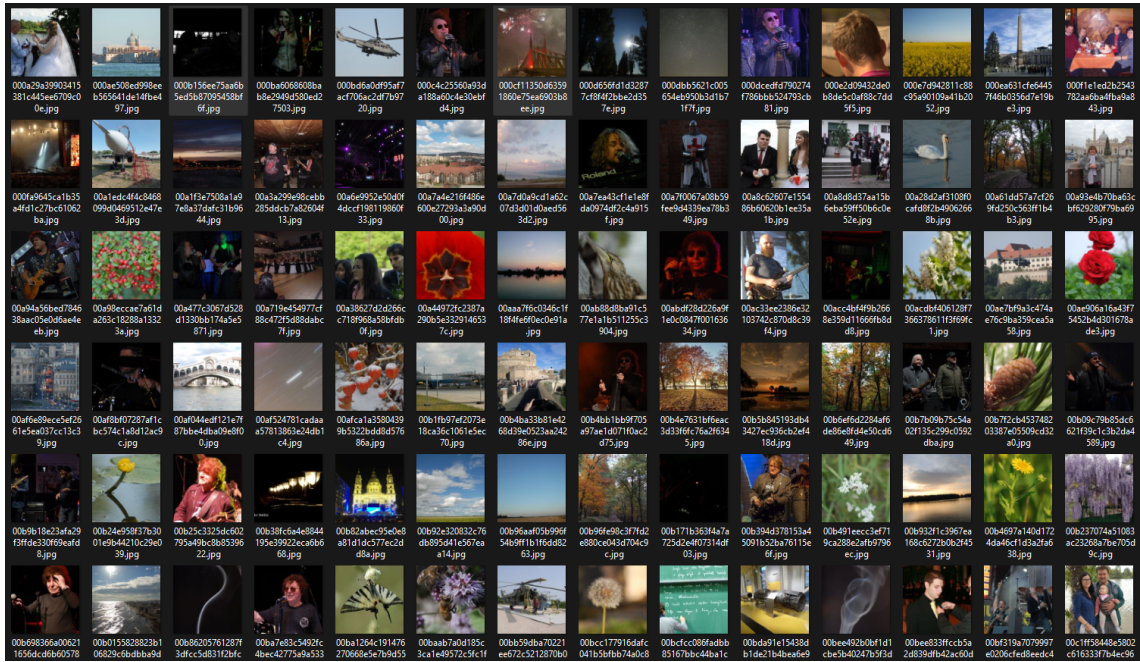
3.1. A forrásképek

A saját képadatbázis összeállításának alapját egy több mint húsz évet felölelő, személyes fotóarchívum képezte, amely 2004-től egészen 2025-ig tartalmaz felvételeket. Ez az anyag tematikailag rendkívül változatos: megtalálhatók benne természetfotók, utazási és városképek, tájképek, légifotók, eseménydokumentációk, koncertek, valamint egyéb élethelyzeteket rögzítő képsorozatok is. Az archívum sokfélesége nemcsak témában, hanem technikai szempontból is igen kiterjedt.

Az évek során több különböző fényképezőgép és eszköztípus került használatba, az egyszerűbb mobiltelefonos kameráktól kezdve a felső kategóriás digitális tükörreflexes (DSLR) fényképezőgépekig. Ennek következtében a képek felbontása, dinamika-tartománya, színmélysége és általános minősége is jelentős változatosságot mutat. A fotók egy része JPEG formátumban, más részük pedig NEF (Nikon RAW) formátumban készült. A NEF fájlok különösen értékesek voltak a feldolgozás során, mivel nyers adatformátumként teljes kontrollt biztosítottak a képfeldolgozás minden lépése felett, és tartalmazták a részletes EXIF metaadatokat is.

Az archívum heterogenitása - amely a képek tartalmi és technikai sokszínűségéből fakad - különösen értékes alapot nyújtott egy erős és jól generalizáló tanítóadatbázis felépítéséhez. A képek eltérő műfajai, kompozíciós megközelítései, fényviszonyai és feldolgozási módjai gazdag vizuális mintázatot biztosítottak, amely elősegítette a modellek robusztus tanulását és a széles körű adaptáció lehetőségét. A forrásanyag sokoldalúsága egyúttal kihívást is jelentett a képek feldolgozása, egységesítése és osztályozása során, amelyet gondosan megtervezett konverziós és előfeldolgozási lépésekkel oldottam meg.

Összességében elmondható, hogy a saját fotóarchívum kiváló alapanyagot biztosított a tanítóadatbázis összeállításához, mivel mennyiségében, témagazdagságában és technikai részletességében egyaránt alkalmas volt arra, hogy a képosztályozási feladathoz megbízható és reprezentatív adatforrást nyújtson.



3.1. ábra. Minta kép a rendezés nélküli kiinduló fotók változatosságáról.

3.1.1. Motiváció

A konvolúciós neurális hálózatok (CNN-ek) hatékony tanításához elengedhetetlen a nagy mennyiségű és kellően változatos adathalmaz megléte. A mesterséges intelligenciával, gépi tanulással és képfeldolgozással foglalkozó irodalom rendszeresen kiemeli, hogy a modell általánosító képessége nagymértékben függ az adatok sokszínűségétől és reprezentativitásától. A jelen munka egyik alapvető célkitűzése egy ilyen

adathalmaz létrehozása volt, amely lehetővé teszi mély neurális hálózatok tanítását különféle vizuális kategóriák felismerésére.

A kiindulási pontot saját, hosszú évek alatt épített fotóarchívumom jelentette, amely 2004-től kezdődően több tízezer képet tartalmaz, a legkülönbözőbb témákban. Ezek között megtalálhatók természetfotók, városképek, utazások, események, koncertek, légi felvételek és sok más egyéb műfaj. A képek változatos eszközökkel - mobiltelefonokkal, kompakt és digitális tükörreflexes fényképezőgépekkel - készültek, ennek megfelelően különböző technikai jellemzőkkel és vizuális stílusokkal rendelkeznek. Ez a heterogenitás ideális alapot biztosított a neurális hálózatok számára szükséges vizuális diverzitás megteremtéséhez.

A saját fotóarchívum felhasználása nemcsak technikai szempontból kínált előnyöket, hanem személyes motivációt is jelentett. A fotózás számomra nem pusztán hobbi, hanem egy olyan kreatív tevékenység, amely szorosan kapcsolódik a vizuális gondolkodásmódomhoz. Mind a grafikai munkák, mind a számítógépes látás területén szerzett tapasztalataim megerősítették bennem azt az elhatározást, hogy a diplomamunka során saját képi világomat és technikai eszköztáramat felhasználva hozzak létre egy értékes és jól strukturált tanítóadatbázist és ezen dolgozzak.

Ez a megközelítés lehetőséget teremtett arra, hogy a személyes érdeklődés és a tudományos célkitűzések szorosan összekapcsolódjanak, valamint hogy az adathalmaz kialakítása ne csupán technikai feladat, hanem egyben kreatív és rendszerszemléletű tervezési folyamat is legyen.

3.1.2. Források

A tanítóadatbázis létrehozásához kizárólag saját készítésű fényképeket használtam fel. A képek az elmúlt húsz év során készültek különféle alkalmakon: kirándulások, utazások, koncertek, repülőnapok, múzeumlátogatások, rendezvények, valamint iskolai és egyetemi projektek keretében. A fotók lefedik a legkülönbözőbb témákat, a természetfotóktól kezdve az urbánus környezeten, eseményfotózáson, építészeti részleteken és tárgyfotókon át egészen a kulturális és technikai kiállításokon készült felvételekig.

Minden kép saját készítésű, a szerzői jog kizárólagosan engem illet. A felvételek minden esetben legálisan, nyilvánosan fotózható helyszíneken vagy külön engedéllyel történtek, teljes mértékben megfelelve az adott helyszínek vonatkozó szabályozá-

sainak. A képekhez tartozó metaadatok (például EXIF-információk) szintén saját eszközeimmel és feldolgozási láncokkal kerültek kinyerésre és megőrzésre.

A forrásként szolgáló képállomány ilyen formában nemcsak jogilag biztonságos és etikus választás, hanem szakmailag is kiváló kiindulópontot jelentett az adatbázisépítéshez, hiszen lehetővé tette a teljes feldolgozási folyamat és az osztályozási szempontok feletti teljes kontrollt.

3.2. A forrásadatbázis elemzése

3.2.1. Darabszámok és méretek

A forrásképeket redundánsan tárolom két darab, egyenként 2 TB kapacitású merevlemezen. A képek strukturált módon, mappákba rendezve kerültek eltárolásra, ahol a mappa neve tartalmazza a készítés dátumát, helyszínét és a témára utaló kulcsszót. Jelenleg a két tároló tartalma nem teljesen azonos, így az átvizsgált és kategorizált képmennyiség 3,01 TB, amely összesen 425 010 fotóból áll.

A képek átlagos mérete körülbelül 7 MB volt. Az adathalmaz feldolgozása során kézi, majd gépi szűrés segítségével eltávolítottam az ismétlődő képeket, így végül 180 750 egyedi fotót azonosítottam. Ezen képek konvertálása, átméretezése, tömörítése és középre vágása után jött létre a végső adatbázis, amely 9,44 GB méretű lett.

Mivel az eredeti képek átlagosan körülbelül 7 MB méretűek voltak, míg az előkészített képek átlagosan 44 kB méretűek, így az összesített adattömörítési arány meghaladja a 60%-ot. Ez a tömörítés jelentős tárhely-megtakarítást jelentett, miközben a neurális hálózatokhoz szükséges releváns vizuális információ megmaradt.

3.2.2. Tematikus kategóriák

A saját fotóarchívum alapján készült adatbázis egyik legfontosabb jellemzője a tartalmi sokszínűség. Az adatok rendkívül heterogén témákból és műfajokból származnak, amely nemcsak a hálózat tanítását támogatja hatékonyan, hanem elősegíti a generalizálhatóbb, robusztusabb modell kialakulását is.

A képek a következő főbb tematikus kategóriákba sorolhatók:

- Természetfotó
- Makrofotó

- Asztrofotó
- Rendezvényfotó
- Koncertek, zenei események
- Repülőfotó és légifotó
- Múzeumok, kiállítások
- Technológiai témák
- Tájképek, városképek
- Absztrakt és kreatív fotók
- Utcai jelenetek, utcaképek
- Portréfotók
- Építészeti felvételek
- Utazási fotók
- Egyéb vegyes témák

A kategorizálás során először automatikus kulcsszavas és képi hasonlóságon alapuló csoportosítást alkalmaztam, amelyet manuális felülvizsgálat és korrekció követett. A kisebb kategóriákat szükség esetén összevontam, az irreleváns vagy nem informatív osztályokat pedig eltávolítottam. A végső osztályozási rendszer így kiegyensúlyozott és jól használható lett a konvolúciós neurális hálózatok tanításához.

3.3. Adatbázis tervezett szerkezete

Ebben a részben szeretném bemutatni, hogy hogyan zajlott az adatbázis tervezése és milyen döntések alapján történt a szerkezet kialakítása.

3.3.1. Képméret és formátum kiválasztása

A képadatbázis egységesítésének egyik kulcsfontosságú lépése a képek méretének és formátumának meghatározása volt. A döntést alapvetően a konvolúciós neurális hálózatok (CNN-ek) működésének követelményei, valamint a számítási teljesítmény és az adathatékonyság szempontjai határozták meg.

A legtöbb klasszikus mélytanulási architektúra, mint például a VGG, ResNet vagy MobileNet, fix bemeneti mérettel dolgozik, amely jellemzően 224×224 pixel. Ez a méret nemcsak kompatibilitási okokból vált szabvánnyá, hanem azért is, mert megfelelő kompromisszumot kínál a részletgazdagság és a feldolgozási igények között. A nagyobb képméretetek ugyan több vizuális információt hordozhatnak, de jelentősen növelik a számítási költségeket, különösen nagy adathalmaz esetén, míg a kisebb képméretetek sok esetben már információvesztéssel járhatnak.

A jelen munkában alkalmazott modellek - mivel előtanított architektúrákra (pl. ResNet) épülnek - kizárólag a szabványos 224×224 pixeles bemenetekkel működnek megfelelően. Ennek megfelelően az adatbázis képeit úgy dolgoztam fel, hogy az eredeti 3:2 képarányú képeket a rövidebb 2 arányú oldal mentén 224px méretűre méreteztem, így a hosszabb oldal 337px méretű lett ezután, középpontos vágással kerültek végleges átméretezésre, így a legtöbb részletet sikerült megtartani torzítás nélkül. Ez a módszer biztosította, hogy a képek vizuálisan kiegyensúlyozottak maradjanak, miközben illeszkedtek a modellek bemeneti követelményeihez.

A képek formátumát tekintve a JPEG lett a választott tömörítési szabvány, mégpedig 80-as minőségi fokozattal. Ez az érték jól kiegyensúlyozza a képminőséget és a tárhelyhasználatot: a tömörítés mértéke nem jár észrevehető minőségromlással, ugyanakkor jelentős mértékben csökkenti a tárolt adatok méretét - különösen több tízezer kép esetén ez gigabájtos nagyságrendű megtakarítást eredményez. A választott tömörítési szint nem befolyásolja negatívan a neurális hálózatok tanulási teljesítményét, ugyanakkor hatékonyabbá teszi az adathalmaz kezelését és betöltését.

A képméret és formátum kiválasztása tehát tudatos döntés eredménye volt, amely figyelembe vette a mélytanulási modellek elvárásait, a rendszer teljesítménykorlátait, valamint az adathatékonyság szempontjait.

Források: [24], [25]

3.3.2. Osztályozási stratégia és kategóriarendszer kialakítása

Az adatbázis osztályozásának tervezése komoly szakmai és gyakorlati kihívást jelentett, mivel a feldolgozott képmennyiség nagyságrendje (180 750 darab, egységesen 224x224 px-es, JPG-80 formátumú, MD5 hash szerint átnevezett fájl) kizárta a kizárólag kézi rendezés lehetőségét. Az automatikus osztályozás alkalmazása elkerülhetlenné vált, azonban a kiválasztott modell(ek) és osztályrendszer(ek) meghatározása során számos szempontot kellett figyelembe vennem.

Elsőként felmerült a kézi kategorizálás lehetősége kisebb mintán, majd annak kiterjesztése, de gyorsan nyilvánvalóvá vált, hogy ekkora adathalmaz esetén ez rendkívül időigényes és gyakorlatilag kivitelezhetetlen lenne. Ezért döntöttem úgy, hogy két különböző, nagy pontosságú, előre betanított osztályozó modell segítségével végzek automatikus kategorizálást, amelyeket utólag kézzel finomhangolok.

A választás szempontjai között szerepelt, hogy a modellek rendelkezzenek nyilvánosan elérhető, nagy léptékű osztályrendszerekre tanított változatokkal (például **Places365** és **ImageNet-1K**), és hogy a kétféle osztályozás eltérő szempontokat képviseljen: míg a **Places365** a helyszínelapú, tematikus megközelítést támogatja, addig az **ImageNet-1K** objektumorientált, tárgyalapú osztályokat kínál.

A kezdeti próbálkozások során a `resnet50_places365` modellt alkalmaztam, amely a helyszínelapú kategorizálásban segített. Az automatikusan generált címkék alapján létrehozott klasztereket kézzel tovább finomítottam, azonban hamar kiderült, hogy a Places365 osztályrendszer mélysége és sokrétűsége (többszintű kategóriák, átfedések, és kevésbé releváns városi vagy beltéri témák) nem illeszkedik jól a saját, főként szabadban készült, természetközpontú fotóimhoz.

Ezért második lépésként az **ImageNet-1K** osztályrendszert választottam, mivel annak egyréteggű, jól definiált objektumközpontú kategóriái jobban illeszkedtek a saját archívumom tematikájához. Ehhez a `convnext_base_ImageNet1K_pretrained` modellt alkalmaztam automatikus osztályozásra, tudatosan elkerülve annak verzióját (**ConvNeXt-Tiny**), amelyet később a saját modelljeim összehasonlító vizsgálatában használtam. Ezzel biztosítani tudtam, hogy az osztályozó modell ne legyen torzító hatással a vizsgált architektúrák teljesítményértékelésére.

Az automatikus osztályozás után az alábbi utómunkákat végeztem el:

- Az üres kategóriák eltávolítása.

- A 300 képnél kevesebbet tartalmazó osztályok képeinek újracsoportosítása más releváns kategóriákba.
- A túl nagy méretű kategóriák esetében gépi (pl. arcfelismerés) és kézi szűrést alkalmaztam az osztályból kilógó, nem illeszkedő képek eltávolítására.
- A teljes adatbázis végső kézi ellenőrzése a címkék konzisztenciájának biztosítása érdekében.

Az eredményül kapott kétféle rendezett adathalmaz közül végül az **ImageNet-1K** szerinti változatot használtam fel a modellképzésekhez és teljesítménymérésekhez. Az **SBphotoThemes180K** nevű saját adatbázis így két külön osztályozási verzióban készült el:

- **Places365 alapú osztályozás:** 121 kategória
- **ImageNet-1K alapú osztályozás:** 114 kategória

A Places365 szerinti változatot archiváltam és megtartottam esetleges jövőbeli kutatások, összehasonlítások vagy kiegészítő elemzések céljaira a méréseket pedig az ImageNet1k szerinti osztályozáson végeztem.

3.3.3. Mappaszerkezet kialakítása

A tanítóadatbázis mappaszerkezetének megtervezése során elsődleges szempont volt a kompatibilitás biztosítása a konvolúciós neurális hálózatok (CNN-ek) tanítására használt modern keretrendszerek (például **PyTorch**, **TensorFlow**, **Keras**) elvárásával. E rendszerek esetében bevett gyakorlat, hogy az adathalmaz fájlstruktúrája tükrözi az osztályozási kategóriákat: a képek a hozzájuk tartozó osztálynevet viselő mappákba kerülnek, és ezek a mappák egy felsőbb szinten — jellemzően **train**, **val**, **test** — három részre osztják az adathalmazt.

Az SBphotoThemes180K adatbázist ennek megfelelően három részre osztottam:

- **Train (80%)** – A tanítóhalmaz, amelyet a modellek tanítására használtam.
- **Validation (10%)** – A validációs halmaz, amelyet a tanulási folyamat közbeni modellértékeléshez alkalmaztam.
- **Test (10%)** – A teszhalmaz, amely a végső, független teljesítményértékelésre szolgált.

A felosztást teljes mértékben automatikusan, véletlenszerű kiválasztással végeztem egy saját fejlesztésű Python szkript segítségével. Ez biztosította, hogy minden kategória képei arányosan és véletlenszerűen kerüljenek szétosztásra a három részhalmaz között, elkerülve a torz reprezentációt és javítva a modell általánosító képességét. A létrehozott könyvtárszerkezet az alábbi logika szerint épül fel:

```
SBphotoThemes180K/
  train/
    kategoria_1/
      fda9ca32e07b59a81de8f98890a26602.jpg
      ...
    kategoria_2/
      700682e68252bfad35426db8ea6c8885.jpg
      ...
    ...
  val/
    kategoria_1/
      39c3811876ac054ec2199710daf7a1ed.jpg
      ...
    ...
  test/
    kategoria_1/
      8111cc7564073f721dbd4376341372bd.jpg
      ...
    ...
```

Ez a struktúra jól illeszkedik a legtöbb betöltő és augmentáló könyvtár (például `torchvision.datasets.ImageFolder`) működéséhez, és lehetővé teszi az adatok gyors és hatékony feldolgozását, illetve a kategóriák közötti világos szétválasztást. A mappák elnevezése megegyezik az ImageNet-1K osztályozás alapján kialakított végleges kategórianévvel, így az adatbázis jól értelmezhető, újrahasznosítható és könnyen bővíthető marad.

3.4. Előfeldolgozás

Az adatbázis szerkezetének és kategóriarendszerének meghatározása után a létrehozás konkrét lépéseinek részletes bemutatása következik. E szakasz az előfeldolgozási fázisra fókuszál, amely során az eredeti képarchívumból kiindulva megtörtént az adatok kiválogatása, átmásolása, előkészítése és az elsődleges konverziók végrehajtása.

3.4.1. Kézi előválogatás és kötegelt adatgyűjtés

Az adatbázis építése az archívum kézi áttekintésével kezdődött, amely során az elmúlt két évtizedben készült fényképekből kiválogattam azokat, amelyek alkalmasnak bizonyultak az osztályozási feladatokhoz. Mivel a teljes fotóarchívum mérete meghaladja a 3 terabájtot, a feldolgozást technikai megfontolások alapján kötegelt formában végeztem.

A fotók az eredeti helyükön két redundáns, 2 TB kapacitású HDD-n helyezkednek el. A képek feldolgozása azonban jelentős számítási igényt támasztott, ezért a képek ideiglenesen egy nagy sebességű SSD-meghajtóra kerültek átmásolásra. A másolás és feldolgozás biztonságos és hatékony végrehajtása érdekében 100 GB méretű adatcsomagokkal dolgoztam.

3.4.2. Méret- és teljesítménykorlátok

A technikai folyamat során fontos szempont volt az adattárolók védelme és a hatékony adatkezelés. Egy 100 GB-os köteg SSD-re másolása körülbelül 30 percet vett igénybe, míg az adott köteg teljes feldolgozása (konverzió, átméretezés, vágás, tömörítés, átnevezés) további nagyjából 15 percet. Így egy köteg teljes feldolgozása körülbelül egy órát igényelt. Ez a kötegelési stratégia lehetővé tette, hogy az adatok gyors elérésű és jól kontrollált környezetben kerüljenek feldolgozásra, anélkül, hogy veszélyeztettem volna az eredeti archívum integritását vagy a merevlemezek épségét.

3.5. Saját feldolgozó futószalag

Már a projekt kezdeti szakaszában egyértelművé vált, hogy az adatok hatékony és konzisztens feldolgozása érdekében szükség van egy automatizált, jól skálázható,

kötegelt feldolgozási rendszerre. A következőkben bemutatom az általam kialakított egyedi „adatfeldolgozó futószalag” felépítését és működését.

3.5.1. Főbb jellemzők és technikai háttér

A rendszer tervezésének egyik alapvető követelménye a párhuzamosan, több mappán is működni képes kötegelt feldolgozás volt, amely lehetővé tette a nagy mennyiségű kép gyors átalakítását és előkészítését.

A feldolgozási folyamatokat egy Windows operációs rendszert futtató, középkeletgóriás teljesítményű munkaállomáson hajtottam végre, amely az alábbi konfigurációval rendelkezett:

- **CPU:** Intel Core i7-10750H (6 mag / 12 szál, 2.6 GHz alapsebességgel)
- **RAM:** 16 GB
- **GPU:** NVIDIA GTX 1650 Ti

A feldolgozás során Windows batch fájlokat és Python szkripteket használtam, amelyek együttműködve végezték el az egyes lépéseket. Fontos megjegyezni, hogy a lokális GPU kapacitása csupán kisebb modellek (pl. ResNet18) tanítására volt alkalmas. Komplexebb modellek futtatása — például ConvNeXt vagy ResNet50 — csak felhőalapú környezetben (Google Cloud, NVIDIA L4 vagy A100 GPU-kon) volt lehetséges. Ezek részleteiről a későbbi fejezetekben számolok be.

3.5.2. A futószalag felépítése és működése

A feldolgozási futószalag az alábbi lépésekből állt, melyeket minden 100 GB-os adatcsomagra külön végrehajtottam:

1. **Forrásfájlok átmásolása:** Az eredeti képek egy többbrétegű könyvtárstruktúrából kerültek átmásolásra SSD-meghajtóra. Ez a lépés egyrészt a redundáns adattárolás biztonságát őrizte meg, másrészt a feldolgozás sebességét jelentősen növelte.
2. **Párhuzamos konverzió:** Egy Windows batch és Python alapú, multimappás feldolgozásra képes, kötegelt párhuzamos szkript átméretezte a képeket a kívánt 224×224 pixeles méretre, és a megfelelő tömörítéssel JPG formátumba konvertálta őket.

3. **MD5 hash alapú átnevezés:** A következő szkript minden képfájlt egyedi MD5 hash alapján nevezett át, így biztosítva a fájlnevek egyediségét és az esetleges duplikátumok kiszűrését.
4. **Képösszegző másolás (sumcopy):** Végül a „sumcopy” nevű szkript összegyűjtötte a feldolgozott képeket, és a végső célmappába másolta azokat a további feldolgozáshoz.

Ez a négy lépés alkotta az előfeldolgozási futószalag első szakaszát, amelyet ismételten végrehajtottam az archívum teljes feldolgozásáig. Az így létrehozott képkészlet már egységes méretű, formátumú és redundanciamentes, de még kategorizálatlan állapotban állt rendelkezésre.

A következő lépésben az adatok automatikus és kézi osztályozása, valamint a végső utófeldolgozás következett. E szakaszokat a következőkben részletezem.

3.5.3. A képfeldolgozó futószalag

Tekintsük most át az előbb megismert képfeldolgozó futószalag elemeit részleteiben.

3.5.4. Konvertálás, Méretezés, Vágás

A képkonverziót egy saját fejlesztésű, `convert.bat + convert_runner.py` nevű szkript végezte, amely Windows batch és Python alapon működő, multimappás kötegelt párhuzamos feldolgozásra képes rendszerként lett kialakítva. Feladata a képek egységes átméretezése 224×224 pixelre, valamint megfelelő tömörítéssel történő konvertálása JPG formátumba.

A folyamat első lépéseként a szkript listázza az adott mappában található képfájlokat, majd minden képre külön feldolgozási folyamatot indít, két lépésben:

- Az **IrfanView** eszközzel történik a nyers képfájlok (pl. NEF formátum) konvertálása és méretezése.
- Ezt követően az **ImageMagick** segítségével történik a középpontos, 1:1 oldalarányú képkivágás.

A két eszköz kombinálására azért volt szükség, mert:

- Az **ImageMagick** nem minden esetben kezelte megbízhatóan a NEF formátumot,

- Az IrfanView viszont nem biztosított precíz középpontos kivágást.

A szkript futtatását egy Python program végezte, amely rekurzívan indította el a `convert.bat` szkriptet minden egyes almappára, lehetővé téve ezzel a több szálon futó, párhuzamos konverziót.

A rendszer teljesítménye rendkívül hatékornak bizonyult: a többmagos CPU-teljesítményt teljes mértékben kihasználta, és magas képfeldolgozási sebességet ért el. Az alábbi táblázatok részletesen bemutatják az egyes műveletek időigényét és a teljes rendszer átlagos teljesítményét.

3.1. táblázat. Képkonverziós műveletek időigénye és optimalizálhatósága

Művelet	Átlagos idő	Lehetőség a gyorsításra?
NEF → JPG konverzió	~35–45 ms	IrfanView optimalizálva
Átméretezés	~10–20 ms	ImageMagick CPU-teljesítményen
Kézi overhead	~2–3 s / futás	Teljesen automatizálható

3.2. táblázat. Átlagos feldolgozási teljesítmény

Sebességmutató	Érték
Képenkénti feldolgozási idő	52,6 ms
Átlagos képfeldolgozási sebesség	~19,0 kép / másodperc
Konverziós sávszélesség (becsült)	~1,27 MB/s (65 KB/kép mellett)
10 000 kép becsült ideje	~10,4 perc
150 000 kép becsült ideje	~2 óra 36 perc

A következő ábra egy mappa feldolgozását mutatja be, amely során a `convert.bat` és a `convert_runner.py` működött együtt a konverziós feladatok elvégzésére:

```
[RUN] D:\WORK\DVD10\Nyaralások 2\Repülőnapok\Vári Gyula és a kalocsai repülőnap 2007\convert.bat
[INFO] Creating 'resized' folder...

[STEP 1] Counting input files...
Found 14 images to process.

[STEP 2] Converting all to resized JPG (short side = 224 px)...
[Converting] 7% (1 / 14)
[Converting] 14% (2 / 14)
[Converting] 21% (3 / 14)
[Converting] 28% (4 / 14)
[Converting] 35% (5 / 14)
[Converting] 42% (6 / 14)
[Converting] 50% (7 / 14)
[Converting] 57% (8 / 14)
[Converting] 64% (9 / 14)
[Converting] 71% (10 / 14)
[Converting] 78% (11 / 14)
[Converting] 85% (12 / 14)
[Converting] 92% (13 / 14)
[Converting] 100% (14 / 14)

Waiting for all IrfanView conversions to finish...

[STEP 3] Cropping to center 224x224 (parallel)...
[Cropping] 7% (1 / 14)
[Cropping] 14% (2 / 14)
[Cropping] 21% (3 / 14)
[Cropping] 28% (4 / 14)
[Cropping] 35% (5 / 14)
[Cropping] 42% (6 / 14)
[Cropping] 50% (7 / 14)
[Cropping] 57% (8 / 14)
[Cropping] 64% (9 / 14)
[Cropping] 71% (10 / 14)
[Cropping] 78% (11 / 14)
[Cropping] 85% (12 / 14)
[Cropping] 92% (13 / 14)
[Cropping] 100% (14 / 14)

Waiting for all cropping to finish...

[DONE] All images processed to 224x224 JPG in 'resized' folder.
```

3.2. ábra. A `convert.bat` és `convert_runner.py` futása egy mappa feldolgozására.

A munkát végző programkód a mellékletben és a GitHub repository-ban olvasható.

3.5.5. Átnevezés és ismétlődések szűrése

A fájlok MD5 hash alapján történő átnevezése kulcsfontosságú lépés volt az adatbázis egységességének és redundancia-mentességének biztosítása érdekében. Mivel minden egyes már konvertált és átméretezett kép a saját MD5 hash értékét kapta fájlnevként, gyakorlatilag kizárhatóvá vált, hogy két azonos kép duplikáltan szerepeljen az adatbázisban.

Ez a művelet azonban a nagy fájlszám és a sok mappa miatt nem volt triviális, és egy hatékony, párhuzamos feldolgozásra képes szkript megírását igényelte. E célra két Python szkriptet fejlesztettem: `rename.py` és `rename2.py`. Ezek a programok rekurzívan végigjárták a teljes mappaszerkezetet, és 50 képenként új szálát indítottak,

így biztosítva a feldolgozás párhuzamosságát. Minden szál az adott képcsoportot hash-elte, majd az eredmény alapján nevezte át őket.

A megközelítés jelentős sebességnövekedést eredményezett, bár még így is kissé lassabbnak bizonyult, mint a konverziós-átméretező szkript, amely a rendszer egyik leggyorsabb komponensének bizonyult.

Egy átnevezési példa:

DSC0123.jpg → 9f6e3148b22b4e13a7b52b66f99fd912.jpg

Az átnevezési szkript futása

A következő ábra egy konkrét futtatást mutat be, ahol a `rename.py` szkript egy mappa képeit hash-eli és nevezi át párhuzamos szálakon keresztül:

```
PS D:\WORK> python .\rename2.py
[INFO] Felderített könyvtárak száma: 440
- .
- DVD04
- DVD04\2009_02
- DVD04\2009_02\kepek mentese 2009 augusztus 1
- DVD04\2009_02\kepek mentese 2009 augusztus 1\resized
- DVD04\2009_02\kepek mentese 2009 augusztus 1      2
- DVD04\2009_02\kepek mentese 2009 augusztus 1      2\resized
- DVD04\2009_02\Képmentés erdőkeresig
- DVD04\2009_02\Képmentés erdőkeresig\resized
- DVD04\2009_02\Mentés a fehértói nyaralásig
- DVD04\2009_02\Mentés a fehértói nyaralásig\resized
- DVD04\2009_02\Mentés Szolnokig
- DVD04\2009_02\Mentés Szolnokig\resized
- DVD04\2009_02\Másolat - Kaposvár-Gesztenyés kert-Pilis
- DVD04\2009_02\Másolat - Kaposvár-Gesztenyés kert-Pilis\resized
- DVD04\2009_02\Nyíregyháza
- DVD04\2009_02\Nyíregyháza\resized
- DVD04\2009_02\resized
- DVD04\2009_02\Telefon 2009
- DVD04\2009_02\Telefon 2009\resized
- DVD04\2009_02\Telefon 2009\Sorozat000
- DVD04\2009_02\Telefon 2009\Sorozat000\resized
- DVD04\2009_02\Telefonképek koncert karácsony
```

3.3. ábra. A `rename.py` szkript futása.

```
[INFO] DVD05\2009\Bolti koncert 2007: 30 fájl átnevezése...
Renaming in DVD05\2009\Bolti koncert 2007: 100%
[DOE] DVD05\2009\Bolti koncert 2007: minden fájl átnevezve.
[INFO] DVD05\2009\Bolti koncert 2007\resized: 30 fájl átnevezése...
Renaming in DVD05\2009\Bolti koncert 2007\resized: 100%
[DOE] DVD05\2009\Bolti koncert 2007\resized: minden fájl átnevezve.
[INFO] DVD05\2009\dunaujváros: 95 fájl átnevezése...
Renaming in DVD05\2009\dunaujváros: 100%
[DOE] DVD05\2009\dunaujváros: minden fájl átnevezve.
[INFO] DVD05\2009\dunaujváros\resized: 95 fájl átnevezése...
Renaming in DVD05\2009\dunaujváros\resized: 100%
[DOE] DVD05\2009\dunaujváros\resized: minden fájl átnevezve.
[INFO] DVD05\2009\Dunaujváros5: 170 fájl átnevezése...
Renaming in DVD05\2009\Dunaujváros5: 100%
[DOE] DVD05\2009\Dunaujváros5: minden fájl átnevezve.
[INFO] DVD05\2009\Dunaujváros5\resized: 170 fájl átnevezése...
Renaming in DVD05\2009\Dunaujváros5\resized: 100%
[DOE] DVD05\2009\Dunaujváros5\resized: minden fájl átnevezve.
[INFO] DVD05\2009\edda_koncert_kiskunmajsa: 25 fájl átnevezése...
Renaming in DVD05\2009\edda_koncert_kiskunmajsa: 100%
[DOE] DVD05\2009\edda_koncert_kiskunmajsa: minden fájl átnevezve.
[INFO] DVD05\2009\edda_koncert_kiskunmajsa\resized: 25 fájl átnevezése...
Renaming in DVD05\2009\edda_koncert_kiskunmajsa\resized: 100%
[DOE] DVD05\2009\edda_koncert_kiskunmajsa\resized: minden fájl átnevezve.
[INFO] DVD05\2009\erdokertes: 60 fájl átnevezése...
Renaming in DVD05\2009\erdokertes: 100%
[DOE] DVD05\2009\erdokertes: minden fájl átnevezve.
[INFO] DVD05\2009\erdokertes\resized: 60 fájl átnevezése...
Renaming in DVD05\2009\erdokertes\resized: 100%
[DOE] DVD05\2009\erdokertes\resized: minden fájl átnevezve.
[INFO] DVD05\2009\Fenykoncert: 34 fájl átnevezése...
Renaming in DVD05\2009\Fenykoncert: 100%
[DOE] DVD05\2009\Fenykoncert: minden fájl átnevezve.
[INFO] DVD05\2009\Fenykoncert\resized: 34 fájl átnevezése...
Renaming in DVD05\2009\Fenykoncert\resized: 100%
[DOE] DVD05\2009\Fenykoncert\resized: minden fájl átnevezve.
[INFO] DVD05\2009\Gesztenyekert: 101 fájl átnevezése...
Renaming in DVD05\2009\Gesztenyekert: 100%
[DOE] DVD05\2009\Gesztenyekert: minden fájl átnevezve.
[INFO] DVD05\2009\Gesztenyekert\resized: 101 fájl átnevezése...
Renaming in DVD05\2009\Gesztenyekert\resized: 100%
[DOE] DVD05\2009\Gesztenyekert\resized: minden fájl átnevezve.
[INFO] DVD05\2009\Gyula: 110 fájl átnevezése...
Renaming in DVD05\2009\Gyula: 100%
[DOE] DVD05\2009\Gyula: minden fájl átnevezve.
[INFO] DVD05\2009\Gyula\resized: 110 fájl átnevezése...
Renaming in DVD05\2009\Gyula\resized: 100%
[DOE] DVD05\2009\Gyula\resized: minden fájl átnevezve.
```

3.4. ábra. A rename.py szkript futása.

Ennek a programnak a kódja is megtekinthető a mellékletben és a GitHub repository-ban.

3.5.6. Az elkészült képek összemásolása

A képfeldolgozó pipeline utolsó lépése az összes .jpg fájl egy mappába másolása amit a sumcopy.py szkript végzett el amiben egy utolsó ismétlődés szűrő mechanizmus is dolgozott.

```
[SKIP] Már létezik: f62ed6c6c9db0c2697bed66e1ba6cb44.jpg
[SKIP] Már létezik: f6607ab3b8732d5e76d3627a7f99c03e.jpg
[SKIP] Már létezik: f68c5aedf4f23b874e36f4d3e54f0201.jpg
[SKIP] Már létezik: f726693d6e7be5fcf0b14037871f1a04.jpg
[SKIP] Már létezik: f8e49a17744c5777804c16ba9a2a4581.jpg
[SKIP] Már létezik: f9002935653f00dae3708e50c3ba042d.jpg
[SKIP] Már létezik: fa999ce85c9f57576946e7fa54c6617e.jpg
[COPY] fc30c6997703cdc53d6894cd0eb653b6.jpg
[SKIP] Már létezik: fc56be0662bd64411282f59796688ce6.jpg
[COPY] fecff9c09ff9273732dd385f3fb98fe8.jpg
[SKIP] Már létezik: ff21626404466274d40d4090ed8a51c4.jpg
[SKIP] Már létezik: ff42656e68f9724a32f15b77122231e9.jpg
[SKIP] Már létezik: ffb249f8b1ba1acbd9c0237f96e9e972.jpg
[SKIP] Már létezik: ffd376ccddbe5cd26963f596424b7daa.jpg
[SKIP] Már létezik: ffe8f5bcd0c0d081d70120e65b4d3630.jpg
[PROCESS] D:\WORK\DVD06\Koncertek_2\2009_2\pestszentimre\resized tartalmának másolása...
[COPY] 00535833fefdb4c846c9f3cfd328c13f.jpg
[COPY] 005fab853fe90b69d71b90e3b61ae082.jpg
[COPY] 08a85c5f3de9f343204f7923911c845a.jpg
[COPY] 08e6ccca30926502ce4a1eed7c231804.jpg
[COPY] 099cec0a9dd1cbb4201fb70a8b06d4f7.jpg
[COPY] 0e31d5f511779cb6d914b4cee83daded.jpg
[COPY] 1095dbcaab940b2e2ff6a9200306f178.jpg
[COPY] 12abcc8f7f1b03b073e416ad73622ee3.jpg
[COPY] 134b56532320fefed93381a5472db3fe.jpg
[COPY] 13659e1f8f024612d25a38a4e5f43b31.jpg
[COPY] 144270eb5abde5ca9d7d954050513c86.jpg
[COPY] 1b2d49fa3089621dbb979a86de7d7692.jpg
[COPY] 1e173026010a2bbbc161365621e12701e.jpg
[COPY] 24d45d646d096ae060d9f5548774eb86.jpg
[COPY] 266e13853a1569405718c001c66010fe.jpg
[COPY] 26b13bc3b7fa53a2ffc7d3fa9fd54d61.jpg
[COPY] 2d98ebf8f92b6bb6952efdaa9d49b292.jpg
[COPY] 331193d30c923311cdc18fec9a468f5f.jpg
[COPY] 3bb5d30fe0b146446460c593f7ea6024.jpg
[COPY] 3eb2c8f57cad9a7d2bfa4e7988e31c1d.jpg
[COPY] 3f8f0ee37c1d0c529645b41ea692f8cf.jpg
[COPY] 405a2c7c24d64b894cbd2a325fa1bbd4.jpg
[COPY] 41a3629a55890dc362767d44eefce48a.jpg
```

3.5. ábra. A sumcopy.py szkript futása.

Ennek a programnak a kódja is megtekinthető a mellékletben és a GitHub repository-ban.

3.6. Kategorizálás és utófeldolgozás

A képadatbázis automatikus kategorizálása a projekt egyik legérdekesebb és legnagyobb számítási igényű lépése volt. Az automatikus osztályozást egy alapos utófeldolgozási folyamat követte, amelynek célja a kategóriák érvényesítése, az irreleváns vagy gyenge csoportosítások kiszűrése, valamint az adatminőség további javítása volt. Az alábbiakban részletesen bemutatom ezt a folyamatot.

3.6.1. Automatikus osztályozás

Az automatikus osztályozás futtatása jelentős számítási kapacitást igényelt, ezért ezt a lépést a Google Colab környezetében, NVIDIA A100 GPU-k segítségével hajtottam

vége. Az osztályozáshoz készített Jupyter notebook-ok elérhetők a GitHub repóban, a fontosabb kódrészletek pedig a mellékletben találhatók.

Két különböző, előképzett modelleszaládra épülő osztályozást végeztem: először a **Places365**, majd az **ImageNet1K** alapján.

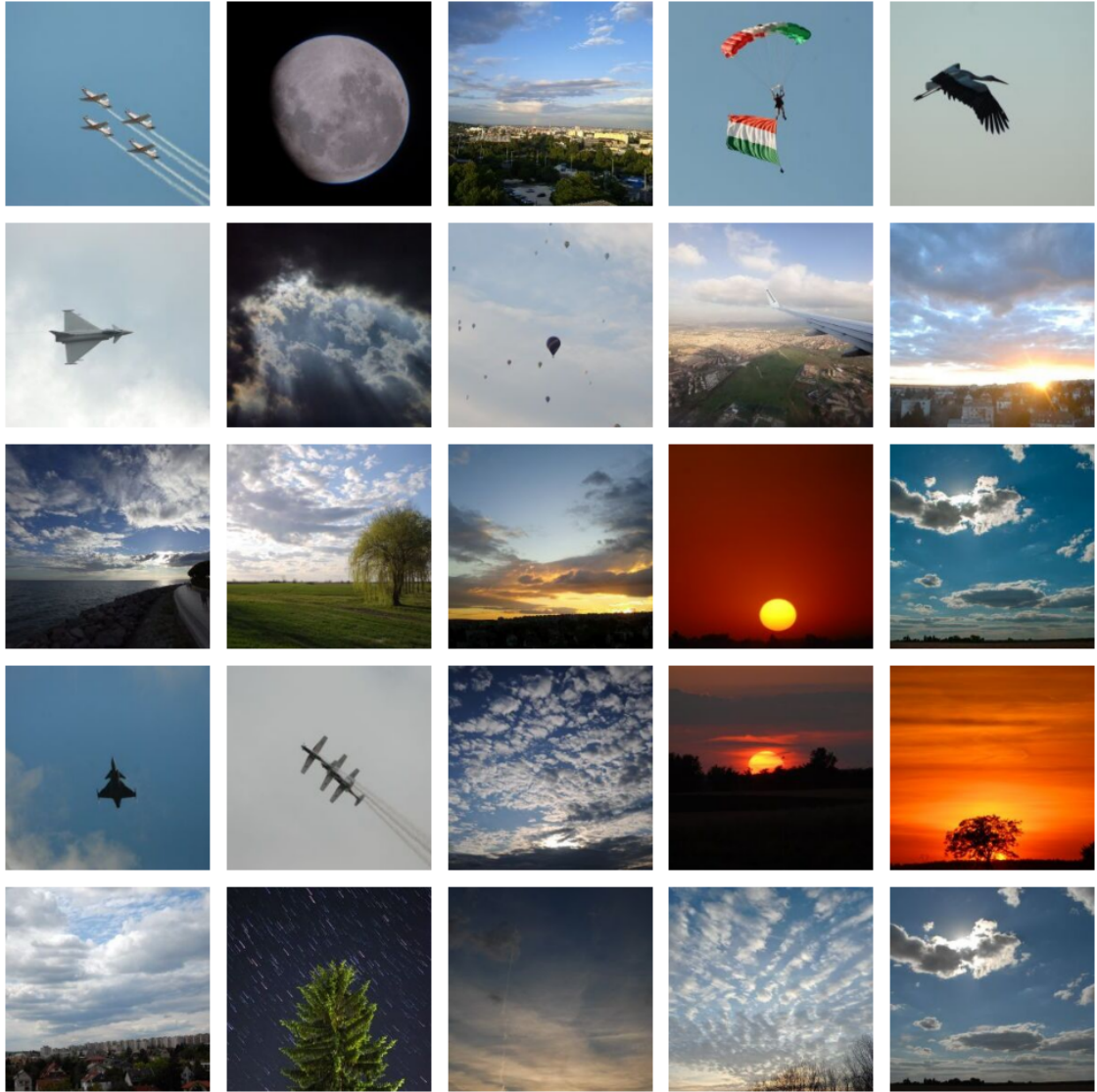
Programkód az autómata szortírozáshoz. (Részlet)

```
1 os.makedirs(sorted_dir, exist_ok=True)
2 class_counts = defaultdict(int)
3 image_map = dict()
4 other_dir = os.path.join(sorted_dir, "other")
5 os.makedirs(other_dir, exist_ok=True)
6
7 image_paths = [os.path.join(unsorted_dir, f) for f in os.listdir(
    unsorted_dir) if f.lower().endswith(".jpg")]
8
9 # Elso kor: top-1 kategoria hozzarendeles
10 print("Elso koros predikcio es ideiglenes hozzarendeles...")
11 for img_path in tqdm(image_paths):
12     img = Image.open(img_path).convert("RGB")
13     inp = transform(img).unsqueeze(0).cuda()
14     with torch.no_grad():
15         out = model(inp)
16         pred_idx = torch.argmax(out, dim=1).item()
17         pred_label = idx_to_label[pred_idx]
18
19     image_map[img_path] = pred_label
20     class_counts[pred_label] += 1
```

3.1. forráskód. Auomatikus ImageNet1k szortírozás - RÉSZLET

3.6.2. Places365 alapú kategorizálás

Elsőként a ResNet50-Places365 modellt használtam, amely főként beltéri és kültéri helyszínek felismerésére specializálódott. A modell 365 különböző helyszínekategoriba sorolja a képeket. Ez a rendezés informatív volt, de a kategóriák gyakran túl specializáltak vagy egymással átfedésben álltak, ami a későbbi feldolgozást nehezítette.



3.7. ábra. Minta a Places365 *sky* osztályából.

3.6.3. ImageNet1K alapú kategorizálás

A második osztályozás során az ImageNet1K osztálykészletet használtam, amely 1000 kategóriát tartalmaz, főként tárgyakra és élőlényekre fókuszálva. Ez a struktúra jobban illeszkedett a saját képi adathalmazom tartalmához, ezért a további kutatásokban ezt az osztályozást használtam elsődlegesen.

Ebben az esetben már beépítettem az automatizált utófeldolgozást is: a szkriptek kiszűrték az üres és a túl kicsi kategóriákat, majd újraosztották a bennük lévő képeket releváns, nagyobb kategóriákba. A végső adathalmaz így kiegyensúlyozottabb és kutatásra alkalmasabb lett.

Az alábbi ábrák véletlenszerű mintákat mutatnak az ImageNet1K osztályozás alap-

ján létrejött kategóriákból:



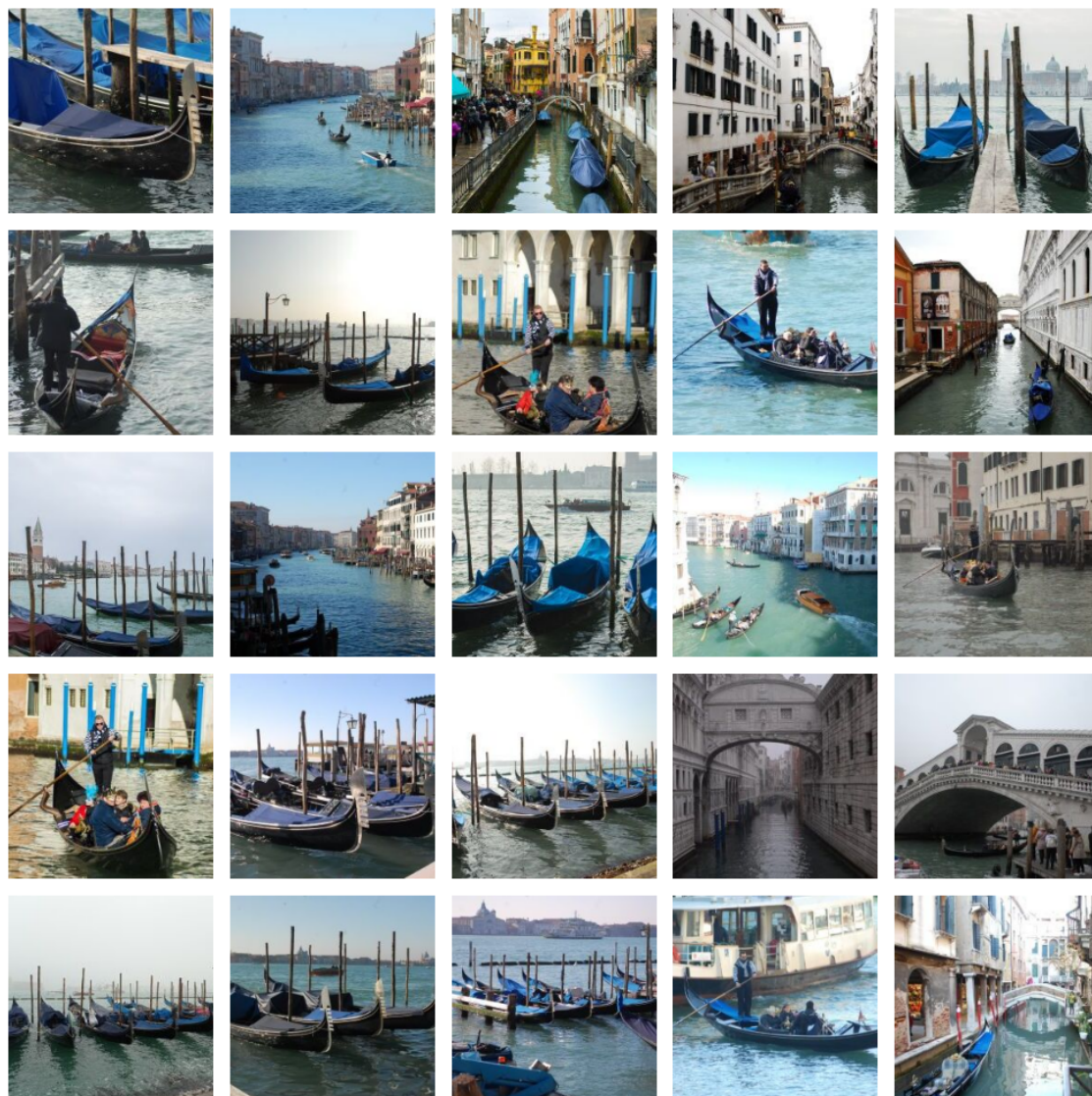
3.8. ábra. Minta az ImageNet1K *airliner* osztályából.



3.9. ábra. Minta az ImageNet1K *bee* osztályából.



3.10. ábra. Minta az ImageNet1K *stage* osztályából.



3.11. ábra. Minta az ImageNet1K *gondola* osztályából.

3.6.4. Adatátviteli nehézségek

Az elkészült adathalmaz mérete megközelítette a 10 GB-ot, és mintegy 180 750 darab kis méretű képfájlt tartalmazott. Az ilyen mértékű, apró fájlokat tartalmazó adathalmaz átvitele komoly technikai kihívást jelentett. Bár Windows rendszeren a fájlkezelés problémamentes volt, a Google Colab felhőalapú környezetébe történő feltöltés meglehetősen körülményesnek bizonyult.

Elsőként a Google Drive használatát próbáltam meg, ám sem a webes, sem az asztali kliens nem tudta hatékonyan kezelni az ekkora számú fájl egyidejű átvitelét. Noha végül sikerült az adathalmazt feltölteni, a Google Drive és a Colab virtuális gépe közötti kapcsolat lassúnak és bizonytalannak bizonyult: a fájlok elérése extrém

hosszú időt vett igénybe, ami gyakorlatilag lehetetlenné tette az adatok közvetlen feldolgozását.

A problémát végül az oldotta meg, hogy az egész adathalmazt ZIP formátumban tömörítettem, és így töltöttem fel a Drive-ra. A Colab környezetben Python szkriptek segítségével másoltam át, majd csomagoltam ki a ZIP-fájlt. Hasonló módszerrel végeztem a kimeneti adatok tömörítését is. Mivel a webes letöltés sem működött stabilan ekkora ZIP-fájlokra, a visszairányú adatmozgatást is Pythonból hajtottam végre: a kimeneti ZIP-fájlt visszamásoltam a Drive-ra, ahonnan az asztali kliens segítségével már megbízhatóan letölthetővé vált.

Ez a tapasztalat jól mutatja, hogy a felhőalapú feldolgozások során nemcsak a számítási teljesítmény, hanem az adatátvitel sebessége és rugalmassága is kritikus szempont lehet.

Ennek a műveletnek a mintakódja lentebb látható.

```
1 # Csatolas
2 drive.mount('/content/drive')
3
4 # utvonalak
5 diploma_path = "/content/drive/MyDrive/diplomamunka"
6 unsorted_zip_path = os.path.join(diploma_path, "unsorted_db.zip")
7 unsorted_dir = "/content/unsorted_db"
8 sorted_dir = "/content/imagenet_sorted_db"
9
10 # Kicsomagolas
11 with zipfile.ZipFile(unsorted_zip_path, 'r') as zip_ref:
12     zip_ref.extractall(unsorted_dir)
13
14 print("Kicsomagolva:", unsorted_dir)
15 # -----
16 print("Tomorites folyamatban...")
17 shutil.make_archive("imagenet_sorted_db", 'zip', sorted_dir)
18
19 print("Visszamasolas a Drive-ba...")
20 shutil.move("/content/imagenet_sorted_db.zip", os.path.join(
21     diploma_path, "imagenet_sorted_db.zip"))
22
23 print("Kesz!")
```

3.2. forráskód. Zip fájlok másolása Drive-ból - RÉSZLET.

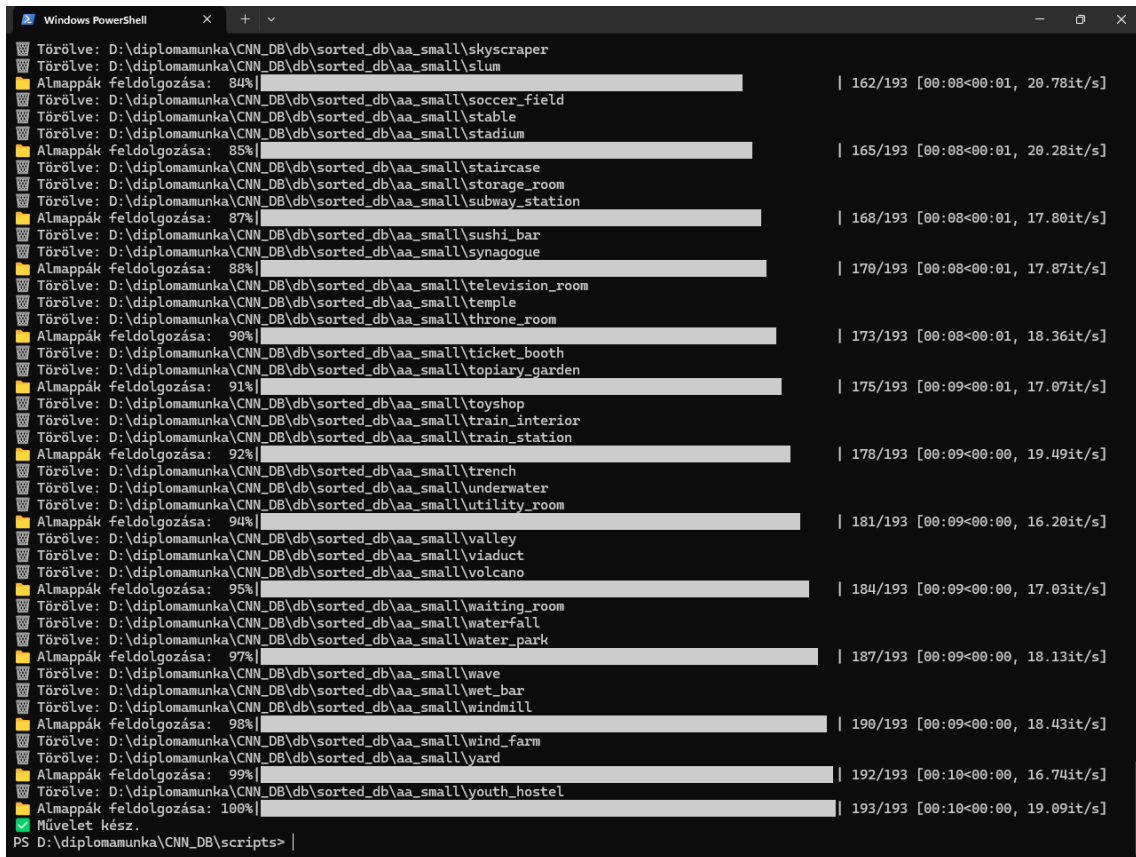
3.6.5. Üres kategóriák eltávolítása

Az osztályozási lépések befejezése után szükségessé vált az üres vagy irrelevánsan kisméretű kategóriák eltávolítása. Erre a célra készítettem el az `empty_deleter.py` nevű segédprogramot, amely automatikusan bejárta a kategóriamappákat, és eltávolította azokat, amelyek nem tartalmaztak elegendő számú képet a további feldolgozáshoz.

3.6.6. Alkategóriák megszüntetése

A Places365 modell kimenete számos alkategóriát is tartalmazott, amelyek nem minden esetben voltak hasznosak, illetve gyakran redundáns vagy nehezen értelmezhető csoportokat eredményeztek. Ezek egybeolvasztására, illetve megszüntetésére külön szkripteket kellett készítenem.

Ezek a `folder-flatten` néven szereplő szkriptek a projekt GitHub-repójában, a `scripts/db/folder_correctors` könyvtárban találhatók. A szkriptek célja az volt, hogy a mélyebb mappaszerkezeteket laposabb, könnyebben kezelhető struktúrába szervezzék. A folyamat részeként külön programokat készítettem az almappák létezésének ellenőrzésére és a redundáns könyvtárstruktúrák azonosítására is.



3.12. ábra. Az alkategóriák megszüntetése

3.6.7. 300 képnél kisebb kategóriák eltávolítása

Az üres kategóriák eltávolítását követően sor került a kevés példányt tartalmazó osztályok szűrésére is. Azokat a kategóriákat, amelyek kevesebb mint 300 képet tartalmaztak, eltávolítottam. Ennek célja az volt, hogy az adatbázis 80-10-10 arányú (Train-Validation-Test) felosztása után is minden halmazban elegendő számú kép maradjon az értelmezhető és hasznos modelltanításhoz. Az újraosztás során figyelembe vettem a leggyakoribb osztályok eloszlását, így a kis kategóriák megszüntetése a teljes osztályspektrum arányait nem torzította jelentősen.

300 képnél kisebb kategóriák újraosztása.

```

1 # Kategoriák szűrese
2 valid_classes = {cls for cls, count in class_counts.items() if
    count >= 300}
3 print(f"Maradt {len(valid_classes)} nagy osztaly.")
4
5 # Ujraosztas: elso kor fajlmozgatas

```



```
6 for img_path, cls in tqdm(image_map.items()):
7     if cls in valid_classes:
8         target_dir = os.path.join(sorted_dir, cls)
9     else:
10        target_dir = other_dir
11    os.makedirs(target_dir, exist_ok=True)
12    shutil.copy2(img_path, os.path.join(target_dir, os.path.
        basename(img_path)))
```

3.3. forráskód. 300 képnél kisebb kategóriák újraosztása.

3.6.8. Kézi adattisztítás, szűrés és törlés

Az automatikus osztályozást követően többnapos, kézi adattisztítási fázis következett, melynek során heurisztikus alapon átvizsgáltam a kategóriákba sorolt képeket. A nem releváns, tévesen besorolt vagy minőségileg nem megfelelő képeket eltávolítottam, hogy a tanítókészlet minősége minél homogénebb és megbízhatóbb legyen. Ez a lépés kiemelkedően fontos volt az adatbázis végső használhatósága szempontjából.

3.6.9. Arcfelismerés alkalmazása

A képek között jelentős számban akadtak portrék vagy embereket ábrázoló jelene-tek, amelyeket az automatikus osztályozás nem minden esetben kezelt megfelelően. Ennek megoldására az OpenCV beépített arcfelismerő algoritmusát alkalmaztam, amellyel automatizáltan kiszűrtem azokat a képeket, amelyek emberi arcot tartal-maztak. Ez nagy segítséget jelentett a kézi szűrés során, különösen az olyan osztá-lyokban, ahol az arctartalom zavaró vagy irreleváns lett volna.

3.6.10. OpenCV arcfelismerő szűrő.

```
1 def move_images_with_faces(folder_path):
2     face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + "
        haarcascade_frontalface_default.xml")
3     faces_dir = os.path.join(folder_path, "faces")
4     os.makedirs(faces_dir, exist_ok=True)
5     image_files = [f for f in os.listdir(folder_path)
6                     if os.path.isfile(os.path.join(folder_path, f))
                        and f.lower().endswith(('.jpg', '.jpeg', '.
                            png'))]
```

```
7     print(f"Vizsgált képek száma: {len(image_files)}")
8     moved_count = 0
9     for filename in tqdm(image_files, desc="Arcok keresése"):
10         file_path = os.path.join(folder_path, filename)
11         try:
12             img = cv2.imread(file_path)
13             if img is None:
14                 continue
15             gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
16             faces = face_cascade.detectMultiScale(gray, scaleFactor
17                 =1.1, minNeighbors=5)
18             if len(faces) > 0:
19                 shutil.move(file_path, os.path.join(faces_dir,
20                     filename))
21                 moved_count += 1
22         except Exception as e:
23             print(f"Hiba a kepnel: {filename} - {e}")
24     print(f"athelyezett képek száma: {moved_count}")
25 #Automatikusan abban a mappában fut, ahol a szkript van
26 if __name__ == "__main__":
27     current_folder = os.path.dirname(os.path.abspath(__file__))
28     print(f"Futtatás helye: {current_folder}")
29     move_images_with_faces(current_folder)
```

3.4. forráskód. OpenCV arcfelismerő szűrő.

3.6.11. Nagy elemszámú kategóriák szétosztása

Egyes kategóriák kiemelkedően nagy elemszámot mutattak, ami torzíthatta volna az osztályeloszlást, és ronthatta volna a modell általánosító képességét. Ezeket a kategóriákat, amennyiben lehetséges volt, kézi heurisztika alapján tovább bontottam altípusokra. Ez a művelet segített az egyensúly fenntartásában, és lehetővé tette a finomabb szemantikai megkülönböztetést.

3.6.12. Felosztás és véglegesítés

Az adatbázis felépítésének utolsó lépése a 80-10-10 arányú Train-Validation-Test felosztás volt. Ennek megvalósításához saját Python szkriptet készítettem, amely garantálta a véletlenszerű, mégis osztályonként kiegyensúlyozott elosztást.

```
PS D:\diplomamunka\CNN_DB\scripts> python .\db_splitter.py
Kategóriák száma: 121
Kategóriák feldolgozása: 100% | 121/121 [02:50<00:00, 1.41s/it]
Képek szétosztva: train / val / test.
Eredmény itt található: D:\diplomamunka\CNN_DB\db\splittd_db
PS D:\diplomamunka\CNN_DB\scripts>
```

3.13. ábra. Az adatbázis felosztása.

A felosztást követően minden készletet ZIP-formátumba tömörítettem, és feltöltöttem az elkészült adatbázisokat a további felhasználás vagy megosztás céljából.

3.7. Kész adatbázis elemzése

3.8. Statisztikák

3.8.1. Darabszámok, méretek

3.8.2. Mappavizsgáló szkript

3.8.3. Kategóriák, statisztikák, méretek

3.8.4. EXIF Statisztika szkript

Tulajdonságok

3.8.5. EXIF statisztika

3.9. Az elkészült adatbázis értékelése

4. fejezet

Modellek és metrikák kiválasztása

4.1. Metodika

4.2. Kiválasztott modellek

4.2.1. ResNet50

4.2.2. DenseNet121

4.2.3. EfficientNet-B0

4.2.4. ConvNeXt-Tiny

4.2.5. ViT-B/16

4.2.6. Inception-v3

4.2.7. NASNet

4.3. A kiválasztott metrikák:

4.3.1. Top-1 pontosság

4.3.2. Top-5 pontosság

4.3.3. Precision

4.3.4. Recall

4.3.5. F1-score

4.3.6. Train és Val loss

4.3.7. Train és Val pontosság

4.4. Összegzés

5. fejezet

Vizsgálati metodika

5.1. I. Fázis: Tanítás nélküli tesztelés

5.2. II. Fázis: Teljes tanítás

5.3. III. Fázis: Iteratív tanítás

5.4. IV. Fázis: Saját modell fejlesztése

6. fejezet

Szoftveres környezet

6.1. Lokális vagy Online

6.2. Google Colab lehetőségek

6.3. Tárolási problémák

6.4. Tanító és tesztelő programok

6.4.1. Tesztelés

6.4.2. Tanítás

6.4.3. Mérés

6.4.4. Problémák és megoldások

Sablon notebook-ok

LaTeX táblázatok

Confusion mátrixok

NASNet optimalizáció

6.5. Összefoglalás

7. fejezet

I. Fázis: Tanítás nélküli tesztelés

7.1. ResNet50

7.2. DenseNet121

7.3. EfficientNet-B0

7.4. ConvNeXt-Tiny

7.5. ViT-B/16

7.6. Inception-v3

7.7. NASNet

7.8. Összefoglaló

8. fejezet

II. Fázis: Teljes tanítás

8.1. ResNet50

8.2. DenseNet121

8.3. EfficientNet-B0

8.4. ConvNeXt-Tiny

8.5. ViT-B/16

8.6. Inception-v3

8.7. NASNet

8.8. Összefoglaló

9. fejezet

III. Fázis: Iteratív tanítás

9.1. ResNet50

9.2. DenseNet121

9.3. EfficientNet-B0

9.4. ConvNeXt-Tiny

9.5. ViT-B/16

9.6. Inception-v3

9.7. NASNet

9.8. Összefoglaló

10. fejezet

IV. Fázis: Saját modell fejlesztése

10.1. A ResNet18

10.2. A Base fej

10.3. A Deep fej

10.4. A Hybrid-v1 fej

10.5. A Hybrid-v2 fej

10.6. Mérések

10.6.1. I. Fázis: Tanítás nélküli tesztelés

10.6.2. II. Fázis: Teljes tanítás

10.6.3. III. Fázis: Iteratív tanítás

10.7. Összefoglalás

11. fejezet

Legfontosabb eredmények

12. fejezet

Összefoglaló

A dolgozat célja a fotó osztályozási probléma vizsgálata volt neurális hálózatokkal saját nagyméretű címkézett adatbázison. A vizsgálat során összegyűjtöttem 180750 egyedi fotót az elmúlt 20 év saját készítésű képeiből, ebből címkézett adatbázist készítettem SBphotothemes180k néven kétféle címkézéssel. Az ImageNet1K címkékből 114 kategória adta az első osztályozást a 300 képnél kisebb osztályok megszüntetése után, a Places365 címkékből 121 kategória adta a második osztályozást. Ezek után 80-10-10 arányban felosztottam tanító, validáló és tesztelő adathalmazra az adatbázist. Az adathalmaz képeit 224*244 méretű négyzetes jpg 80 tömörítésű képekké konvertáltam. Így a közel 2TB mennyiségű forrásképből nagyjából 10GB méretű adatbázis készült. Az elkészült adatbázison vizsgáltam az osztályozási eloszlásokat, és az EXIF adatokat. Az osztályozások estén az átlagos osztályméret 1500 kép körül alakult, a medián 1000 kép körül alakult ami jó, de mivel természetes adatokról beszélünk, így az aszimmetria magas, 8.4 és a szórás is magas 3200 körüli tehát tipikus Long-tail eloszlást mutat. Az EXIF adatok konverzió közbeni megőrzése és elemzése különösen fontos volt a részletes statisztikák elvégzéséhez és a későbbi vizsgálatok lehetőségeinek biztosításához. Az így elkészített adatbázis segítségével három fázisos vizsgálatot végeztem hét ismert és gyakran használt modellen. Ezek a modellek a ResNet50, DenseNet121, EfficientNet-B0, ConvNeXt-Tiny, ViT-B/16, Inception-v3, NASNet voltak. Az alkalmazott metrikák a következők lettek: Top-1 és Top-5 pontosság, Precision, Recall, F1-score. Ezeken túl vizsgáltam a tanítás közbeni validációs és tanítási pontosságokat. Mindhárom vizsgálatot ugyanazon az adathalmazon végeztem, de más-más tanítóhalmazzal tanítottam. A három vizsgálati fázis a tanítás nélküli tesztelés, majd a teljes tanítás utáni vizsgálat volt, végül az iteratívan

növekvő tanítóhalmaz vizsgálata következett. A vizsgálat első lépése a tanítás nélküli modellek tesztelése volt a beépített súlyokkal. Ekkor a modellek 50% körüli Top 1 és 80% körüli Top 5 pontosságot és 40% körüli Precision-t nyújtottak. Itt a ConvNeXt-Tiny és ViT-B/16 emelkedtek ki. A második fázis során a teljes 144000 képes tanítóhalmazon tanítottam minden modellt 10 epoch-on keresztül. Ekkor a modellek 75% körüli Top 1 és 90% körüli Top 5 pontosságot és 70% körüli Precision-t nyújtottak a EfficientNet-B0 és NASNet emelkedtek ki. Az utolsó vizsgálat során mindig a tanítatlan modellről indulva 1000, 5000, 10000, 25000 képes tanítóhalmazon vizsgáltam a modelleket szintén 10 epoch-on keresztül. Az adataink alapján a ConvNeXt-Tiny és az EfficientNet-B0 modellek kiemelkedtek 25000 mintaszámmal történő tanítás után, míg a ResNet50 és DenseNet121 modellek gyengébben teljesítettek az alacsonyabb mintaszámokon, de növekvő mintaszámmal jelentős javulást mutattak. Eközben az Inception-v3 és a ViT-B/16 pedig a kis adathalmazon mutatott pontosság terén emelkedtek ki. Ezután megkíséreltem elkészíteni egy kis méretű, lokális alkalmazásra optimalizált erőforrástakarékos modellt a ResNet18-ra építve egy saját fejrész kifejlesztésével. Ennek során 4 modellt készítettem, egy egyrétegű, egy mély, és két modern fejrésszel rendelkezőt. Ezen a négy modellen is elvégeztem a fent ismertetett három mérési fázist. Az egyrétegű fej meglepően jól teljesített, a mély fejrész viszont jelentősen alulmaradt a fenti metrikákban, a modern fejrész csak kicsit nyújtott nagyobb pontosságot, mint az egyrétegű, de jelentősen gyorsabban futott. Összességében a kifejlesztett modell csak kis mértékben maradt alul a vizsgált hatalmas modellektől, de futási teljesítménye sokkal jobb azokénál így nem pontosságkritikus, de erőforráskímélő megoldásokhoz, például hordozható eszközök számára ideális. Összességében a nagyméretű adathalmaz, a többféle osztályozás, a nagy mennyiségű mért adat és a saját modellek további vizsgálatok alapját képezhetik a jövőben.

13. fejezet

További lehetőségek

14. fejezet

Befejezés

Irodalomjegyzék

- [1] Neha Sharma, Vibhor Jain és Anju Mishra. „An Analysis Of Convolutional Neural Networks For Image Classification”. *Procedia Computer Science* 132 (2018. jan.), 377–384. old. DOI: 10.1016/j.procs.2018.05.198.
- [2] Yilin Hou. „Photo Content Classification Using Convolutional Neural Network”. *Journal of Physics: Conference Series* 1651.1 (2020. nov.), 12179. old. DOI: 10.1088/1742-6596/1651/1/012179. URL: <https://dx.doi.org/10.1088/1742-6596/1651/1/012179>.
- [3] Alfonso Ramos Michel és tsai. *Image Classification with Convolutional Neural Networks*. 2021. júl., 445–473. old. ISBN: 978-3-030-70541-1. DOI: 10.1007/978-3-030-70542-8_18.
- [4] Dan Hendrycks és Kevin Gimpel. „Gaussian Error Linear Units (GELUs)”. *arXiv preprint arXiv:1606.08415* (2016).
- [5] Jimmy Lei Ba, Jamie Ryan Kiros és Geoffrey E Hinton. „Layer Normalization”. *arXiv preprint arXiv:1607.06450* (2016).
- [6] Nitish Srivastava és tsai. „Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. *Journal of Machine Learning Research*. 15. köt. 2014, 1929–1958. old.
- [7] Zewen Li és tsai. „A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects”. *arXiv preprint arXiv:2004.02806* (2020).
- [8] Jia Deng és tsai. „ImageNet: A Large-Scale Hierarchical Image Database”. *2009 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE. 2009, 248–255. old.
- [9] Bolei Zhou és tsai. „Places: A 10 million Image Database for Scene Recognition”. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2017, 1452–1460. old.

- [10] Laith Alzubaidi és tsai. „Review of deep learning: concepts, CNN architectures, challenges, applications, future directions”. *Journal of Big Data* 8.1 (2021), 53. old.
- [11] Stéphane Mallat. „Understanding Deep Convolutional Networks”. *arXiv preprint arXiv:1601.04920* (2016).
- [12] MLCommons Algorithms Working Group. *Benchmarking Neural Network Training Algorithms*. <https://arxiv.org/html/2306.07179>. 2023.
- [13] Kaiming He és tsai. „Deep Residual Learning for Image Recognition”. *arXiv preprint arXiv:1512.03385* (2015).
- [14] Gao Huang és tsai. „Densely Connected Convolutional Networks”. *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, 4700–4708. old.
- [15] Mingxing Tan és Quoc V Le. „EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks”. *International Conference on Machine Learning*. PMLR. 2019, 6105–6114. old.
- [16] Zhuang Liu és tsai. „A ConvNet for the 2020s”. *arXiv preprint arXiv:2201.03545* (2022).
- [17] Alexey Dosovitskiy és tsai. „An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. *arXiv preprint arXiv:2010.11929* (2020).
- [18] Christian Szegedy és tsai. „Rethinking the Inception Architecture for Computer Vision”. *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, 2818–2826. old.
- [19] Barret Zoph és tsai. „Learning Transferable Architectures for Scalable Image Recognition”. *arXiv preprint arXiv:1707.07012* (2017).
- [20] Sergey Ioffe és Christian Szegedy. „Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”. *Proceedings of the 32nd International Conference on Machine Learning*. 2015, 448–456. old.
- [21] Kaiming He és tsai. „Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition”. *European Conference on Computer Vision*. 2014, 346–361. old.

- [22] Y. Zhang, S. Wang és Y. Liu. „A Survey on Deep Transfer Learning and Beyond”. *Mathematics* 10.19 (2022), 3619. old.
- [23] Nuttapoom Amornpashara és tsai. „Landscape photo classification mechanism for context-aware photography support system”. *2015 IEEE International Conference on Consumer Electronics (ICCE)*. 2015, 663–666. old. DOI: 10.1109/ICCE.2015.7066570.
- [24] Mats L. Richter és tsai. „(Input) Size Matters for CNN Classifiers”. *Artificial Neural Networks and Machine Learning – ICANN 2021*. Springer International Publishing, 2021, 133–144. old. ISBN: 9783030863401. DOI: 10.1007/978-3-030-86340-1_11. URL: http://dx.doi.org/10.1007/978-3-030-86340-1_11.
- [25] Hossein Talebi és Peyman Milanfar. *Learning to Resize Images for Computer Vision Tasks*. 2021. arXiv: 2103.09950 [cs.CV]. URL: <https://arxiv.org/abs/2103.09950>.

Ábrák jegyzéke

3.1. Minta kép a rendezés nélküli kiinduló fotók változatosságáról.	22
3.2. A <code>convert.bat</code> és <code>convert_runner.py</code> futása egy mappa feldolgozása.	34
3.3. A <code>rename.py</code> szkript futása.	35
3.4. A <code>rename.py</code> szkript futása.	36
3.5. A <code>sumcopy.py</code> szkript futása.	37
3.6. Minta a <code>Places365 airfield</code> osztályából.	39
3.7. Minta a <code>Places365 sky</code> osztályából.	40
3.8. Minta az <code>ImageNet1K airliner</code> osztályából.	41
3.9. Minta az <code>ImageNet1K bee</code> osztályából.	42
3.10. Minta az <code>ImageNet1K stage</code> osztályából.	43
3.11. Minta az <code>ImageNet1K gondola</code> osztályából.	44
3.12. Az alkategóriák megszüntetése	47
3.13. Az adatbázis felosztása.	50

Táblázatok jegyzéke

3.1. Képkonverziós műveletek időigénye és optimalizálhatósága	33
3.2. Átlagos feldolgozási teljesítmény	33

Forráskódjegyzék

3.1. Auomatikus ImageNet1k szortírozás - RÉSZLET	38
3.2. Zip fájlok másolása Drive-ból - RÉSZLET.	45
3.3. 300 képnél kisebb kategóriák újraosztása.	47
3.4. OpenCV arcfelismerő szűrő.	48